IIT ROORKEE

NPTEL ONLINE CERTIFICATION COURSE

Confusion matrix and ROC - I

Dr A. RAMESH

DEPARTMENT OF MANAGEMENT STUDIES

Agenda

- Confusion matrix
- Receiver operating characteristics curve

Why Evaluate?

- Multiple methods are available to classify or predict
- For each method, multiple choices are available for settings
- To choose best model, need to assess each model's performance

Accuracy Measures (Classification)

Misclassification error

(0, 1)

- Error = classifying a record as belonging to one class when it belongs to another class.

- Error rate = percent of misclassified records out of the total records in the validation data

Confusion Matrix

Classification Confusion Matrix		
	Predicted Class	
Actual Class	1	0
1	201	85
0	25	2689

201 1's correctly classified as "1"

85 1's incorrectly classified as "0"

25 0's incorrectly classified as "1"

2689 0's correctly classified as "0"

Error Rate

Classification Confusion Matrix		
	Predicted Class	
Actual Class	1	0
1	201	85
0	25	2689

Overall error rate = (25+85)/3000 = 3.67%

Accuracy = 1 – err = (201+2689) = 96.33%

If multiple classes, error rate is:

(sum of misclassified records)/(total records)

Cutoff for classification

Most algorithms classify via a 2-step process:

For each record,

1. Compute **probability of belonging to class "1"**
2. Compare to cutoff value, and classify accordingly

- Default cutoff value is 0.50
  - If >= 0.50, classify as "1"
  - If < 0.50, classify as "0"
- Can use different cutoff values
- Typically, error rate is lowest for cutoff = 0.50

Cutoff Table

Actual Class	Prob. of "1"	Actual Class	Prob. of "1"
1	0.996	1	0.506
1	0.988	0	0.471
1	0.984	0	0.337
1	0.980	1	0.218
1	0.948	0	0.199
1	0.889	0	0.149
1	0.848	0	0.048
0	0.762	0	0.038
1	0.707	0	0.025
1	0.681	0	0.022
1	0.656	0	0.016
0	0.622	0	0.004

- If cutoff is 0.50: 11 records are classified as “1”
- If cutoff is 0.80: seven records are classified as “1”

Confusion Matrix for Different Cutoffs

Cut off Prob.Val. for Success (Updatable)	0.25

Classification Confusion Matrix		
	Predicted Class	
Actual Class	owner	non-owner
owner	11	1
non-owner	4	8

Cut off Prob.Val. for Success (Updatable)	0.75

Classification Confusion Matrix		
	Predicted Class	
Actual Class	owner	non-owner
owner	7	5
non-owner	1	11

Compute Outcome Measures

Confusion Matrix:

	Predicted Class = 0	Predicted Class = 1
Actual Class = 0	True Negatives (TN)	False Positives (FP)
Actual Class = 1	False Negatives (FN)	True Positives (TP)

N = number of observations

Overall accuracy = (TN + TP)/N

Overall error rate = (FP + FN)/N

Sensitivity = TP/(TP + FN)

False Negative Error Rate = FN/(TP + FN)

Specificity = TN/(TN + FP)

False Positive Error Rate = FP/(TN + FP)

When One Class is More Important

In many cases it is more important to identify members of one class

- Tax fraud
- Credit default
- Response to promotional offer
- Detecting electronic network intrusion
- Predicting delayed flights

In such cases, we are willing to tolerate greater overall error, in return for better identifying the important class for further attention

ROC curves

- *ROC = Receiver Operating Characteristic*
- Started in electronic signal detection theory (1940s - 1950s)
- Has become very popular in biomedical applications, particularly radiology and imaging
- Also used in machine learning applications to assess classifiers
- Can be used to compare tests/procedures

ROC curves: simplest case

- Consider diagnostic test for a disease
- Test has 2 possible outcomes:
  - ‘positive’ = suggesting presence of disease
  - ‘negative’
- An individual can test either positive or negative for the disease

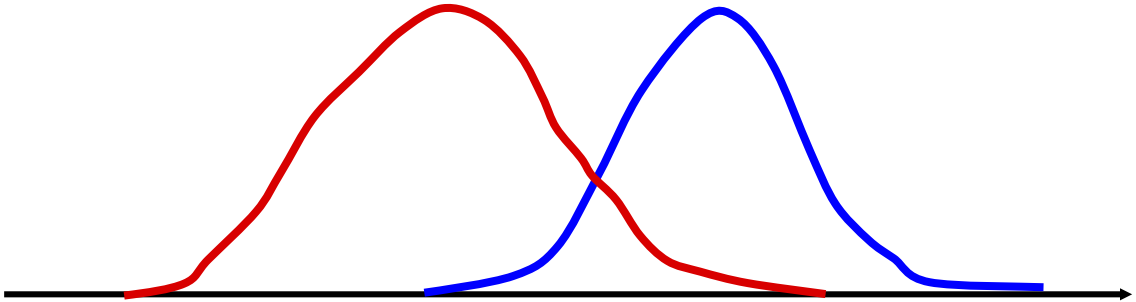
ROC Analysis

- True Positives = Test states you have the disease when you do have the disease
- True Negatives = Test states you do not have the disease when you do not have the disease
- False Positives = Test states you have the disease when you do not have the disease
- False Negatives = Test states you do not have the disease when you do

Specific Example

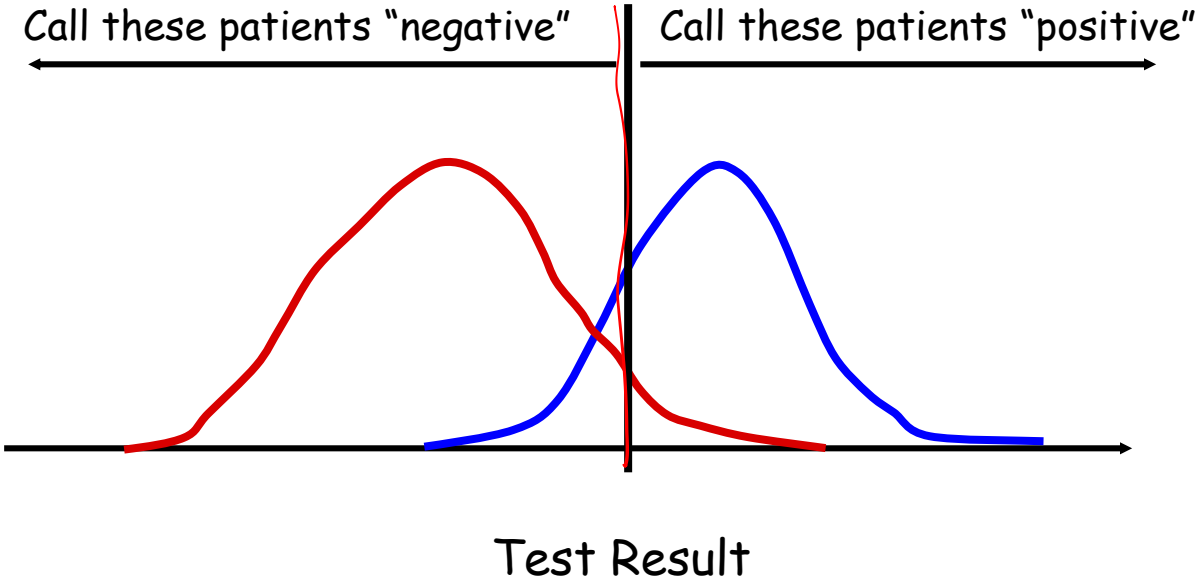
Patients without the disease

Patients with disease

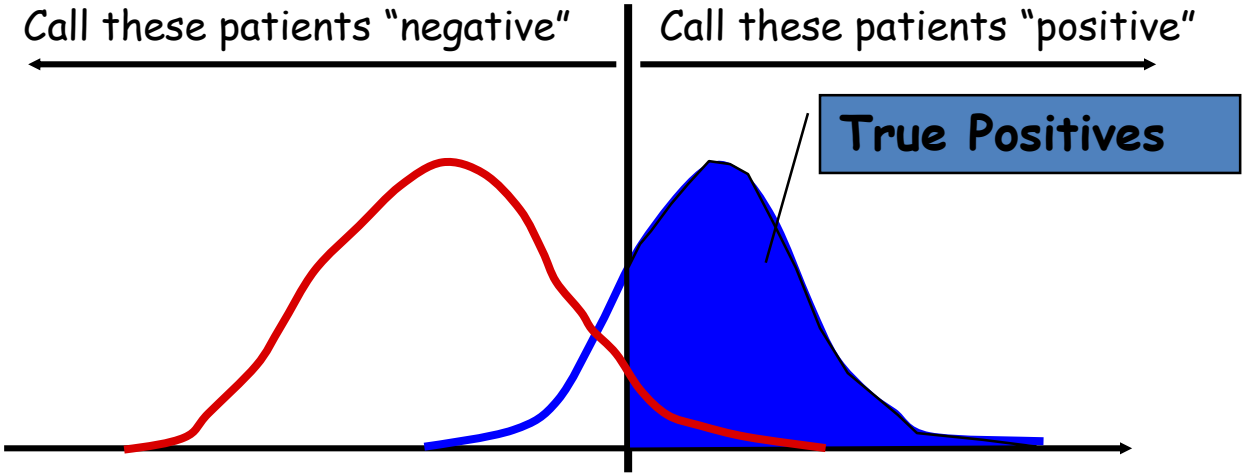


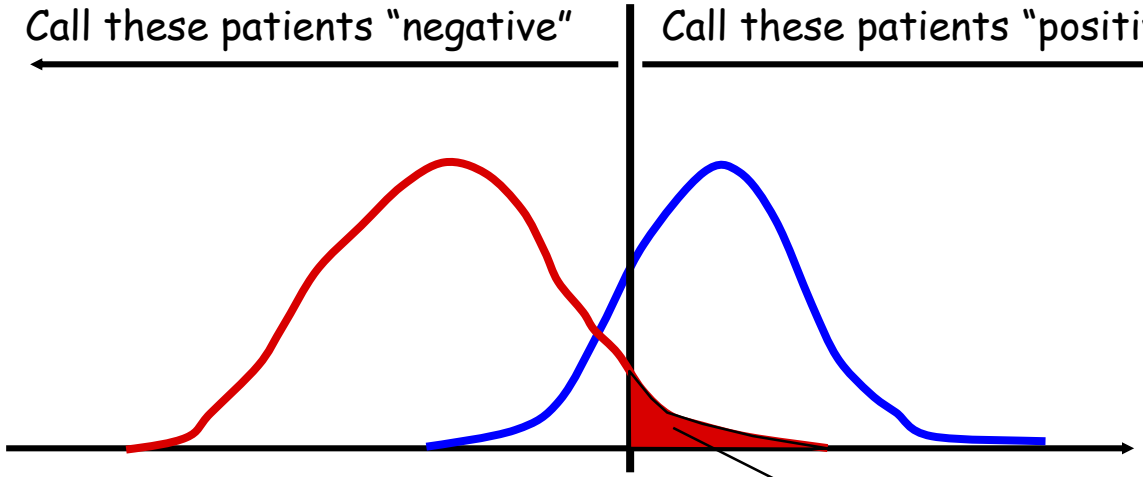
Test Result

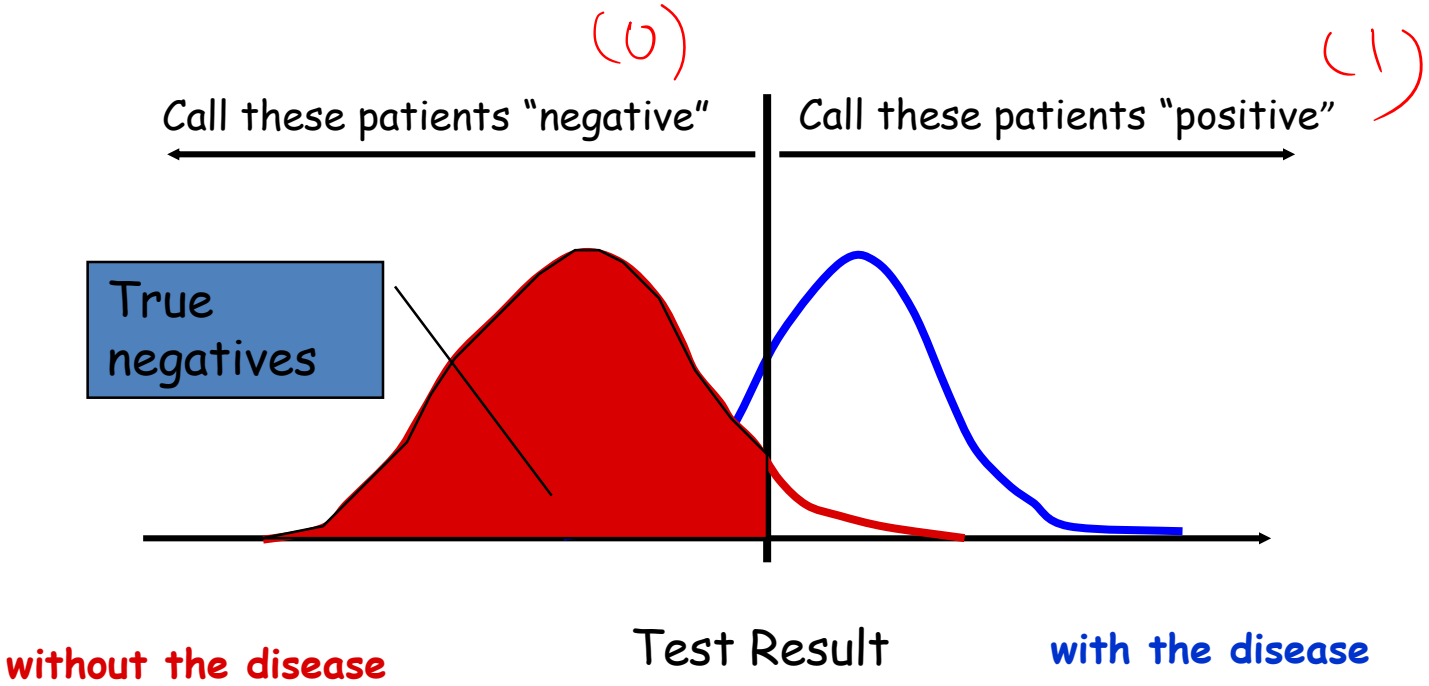
Threshold

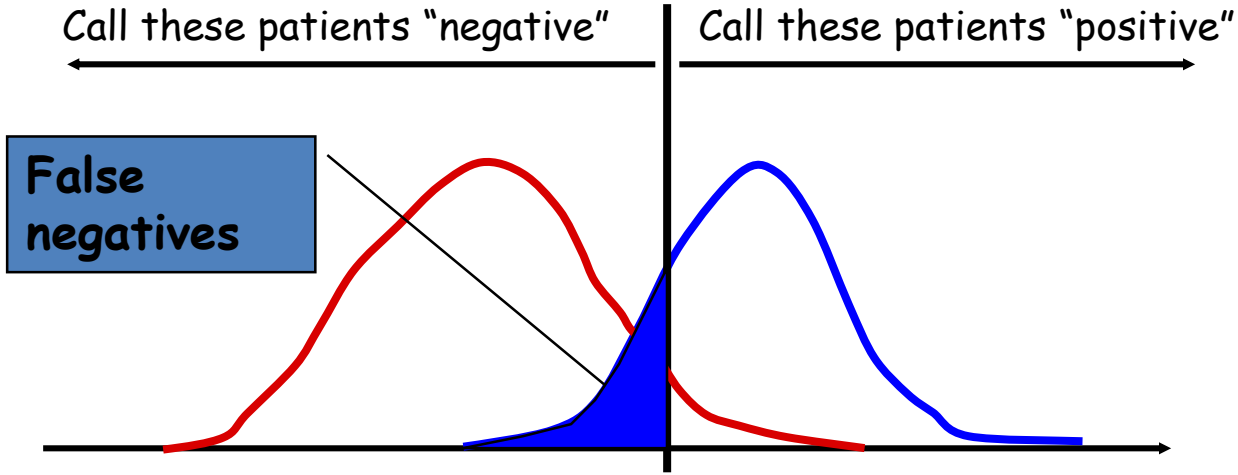

Call these patients "negative"
Call these patients "positive"
Test Result

Some definitions ...

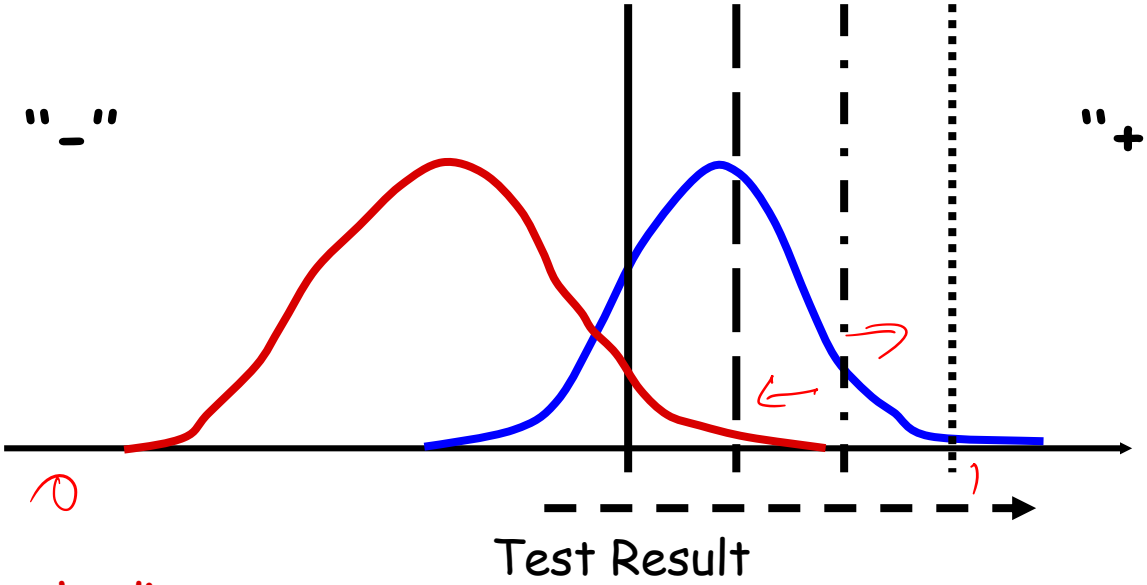



Call these patients "negative"
Call these patients "positive"
with the disease
without the disease
Test Result
False Positives


(0)
(1)
Call these patients "negative"
Call these patients "positive"
True negatives
without the disease
Test Result
with the disease


Call these patients "negative"
Call these patients "positive"
False negatives
Test Result
without the disease
with the disease

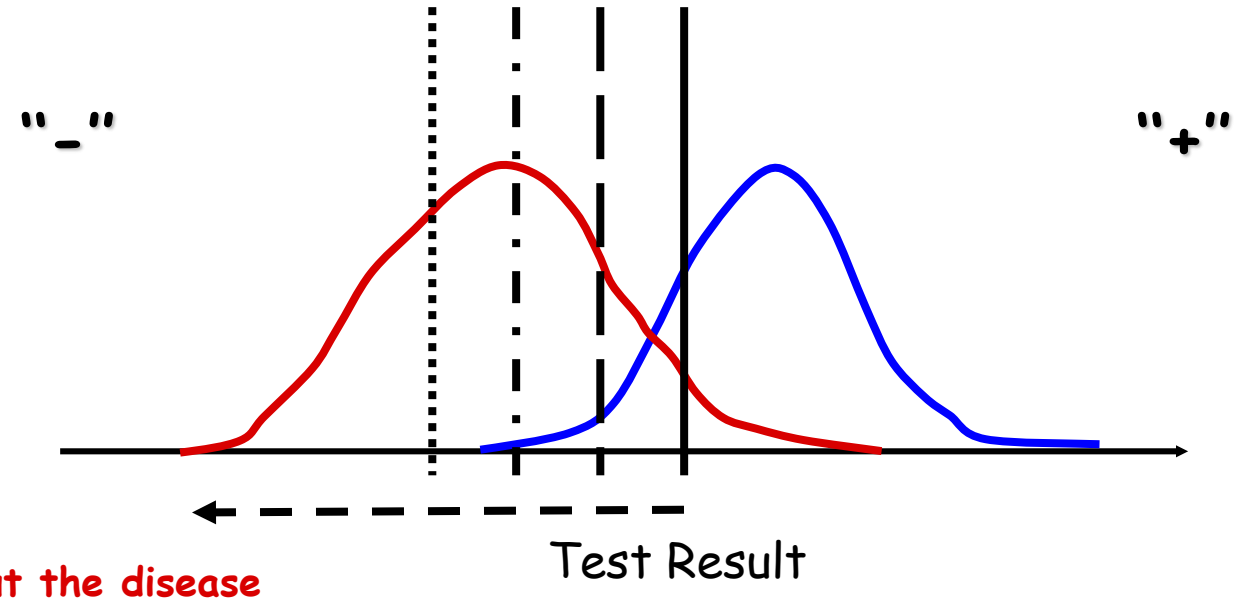
Moving the Threshold: right



without the disease

with the disease

Moving the Threshold: left



without the disease

with the disease

Threshold Value

- The outcome of a logistic regression model is a probability
- Often, we want to make a binary prediction
- We can do this using a *threshold value* t
- If $P(y = 1) \geq t$, predict positive
  - If $P(y = 1) < t$, predict negative
  - What value should we pick for t?

Threshold Value

- Often selected based on which errors are “better”
- If t is **large**, predict positive rarely (when P(y=1) is large)
  - More errors where we say negative , but it is actually positive
  - Detects patients who are negative
- If t is **small**, predict negative rarely (when P(y=1) is small)
  - More errors where we say positive, but it is actually negative
  - Detects all patients who are positive
- With no preference between the errors, select t = 0.5
  - Predicts the more likely outcome

Selecting a Threshold Value

- Compare actual outcomes to predicted outcomes using a *confusion matrix (classification matrix)*

	Predicted = 0	Predicted = 1
Actual = 0	True Negatives (TN)	False Positives (FP)
Actual = 1	False Negatives (FN)	True Positives (TP)

True disease state vs. Test result

Test / Disease	not rejected/accepted	rejected
No disease (D = 0)	☺ specificity	X Type I error (False +) α
Disease (D = 1)	X Type II error (False -) β	☺ Power $1 - \beta$; sensitivity

Classification matrix: Meaning of each cell

Actual Class	Predicted Class: C_0	Predicted Class: C_1
C_0	$n_{0,0}$ = number of C_0 cases classified correctly	$n_{0,1}$ = number of C_0 cases classified incorrectly as C_1
C_1	$n_{1,0}$ = number of C_1 cases classified incorrectly as C_0	$n_{1,1}$ = number of C_1 cases classified correctly

Alternate Accuracy Measures

If "C_1" is the important class,

Sensitivity = % of "C_1" class correctly classified

$$\textbf{Sensitivity} = n_{1,1} / (n_{1,0} + n_{1,1})$$

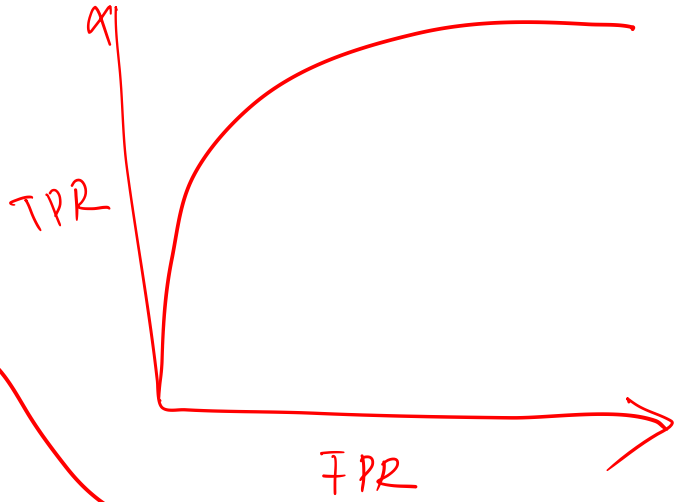
Specificity = % of "C_0" class correctly classified

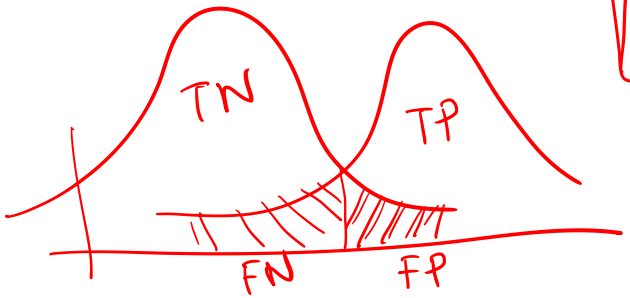
$$\textbf{Specificity} = n_{0,0} / (n_{0,0} + n_{0,1})$$

- **False positive rate** = % of predicted "C_1's" that were not "C_1's"
- **False negative rate** = % of predicted "C_0's" that were not "C_0's"

Receiver Operator Characteristic (ROC) Curve

- True positive rate (sensitivity) on y-axis
  - Proportion of positive
- False positive rate (1-specificity) on x-axis
  - Proportion of negative labelled as positive
- **Low Threshold**
  - Low specificity
  - High sensitivity





Selecting a Threshold using ROC

- Captures all thresholds simultaneously
- **High threshold**
  - High specificity
  - Low sensitivity
- **Low Threshold**
  - Low specificity
  - High sensitivity

Thank You

IIT ROORKEE

NPTEL ONLINE CERTIFICATION COURSE

Confusion Matrix and ROC-II

Dr A. RAMESH

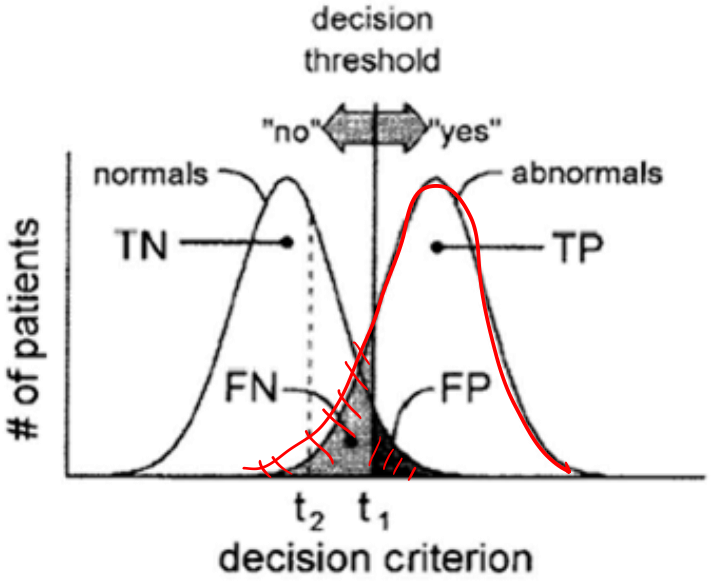
DEPARTMENT OF MANAGEMENT STUDIES

Agenda

- Receiver operating characteristics curve
- Optimum threshold value

ROC analysis

- True Positive Fraction
  - TPF = TP / (TP+FN)
  - also called *sensitivity*
  - true abnormals called abnormal by the observer
- False Positive Fraction
  - FPF = FP / (FP+TN)
- *Specificity* = TN / (TN+FP)
  - True normals called normal by the observer
  - FPF = 1 - specificity

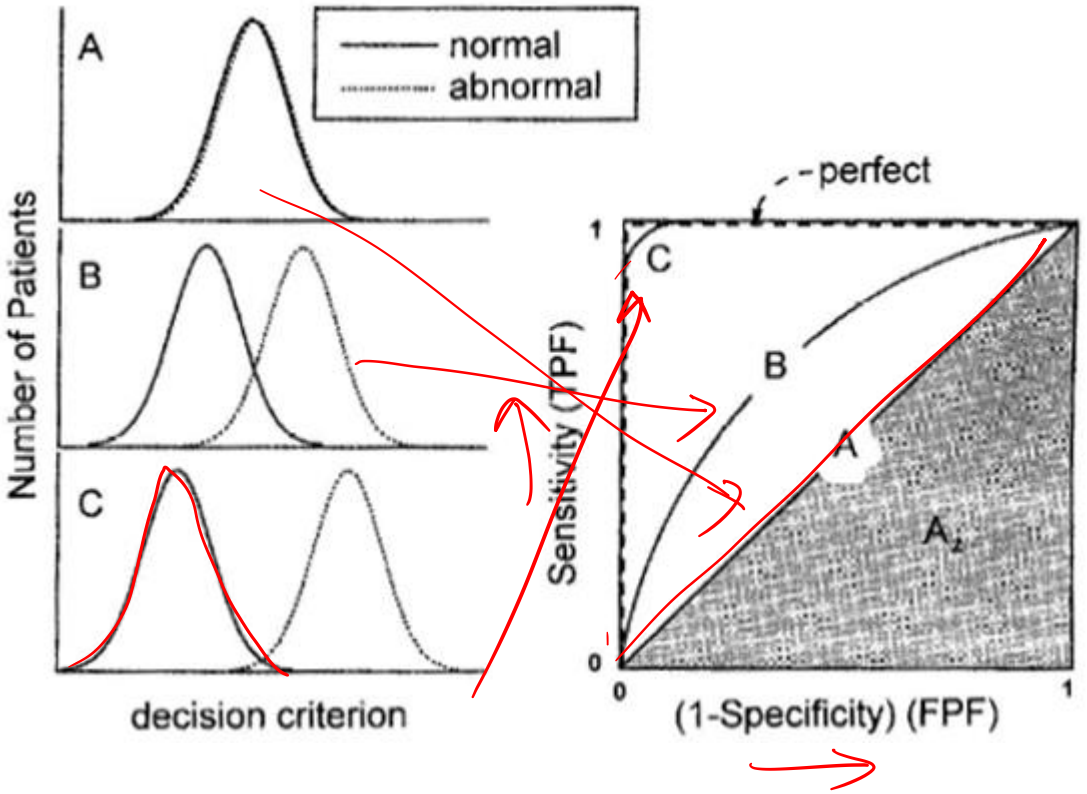


Evaluating classifiers (via their ROC curves)

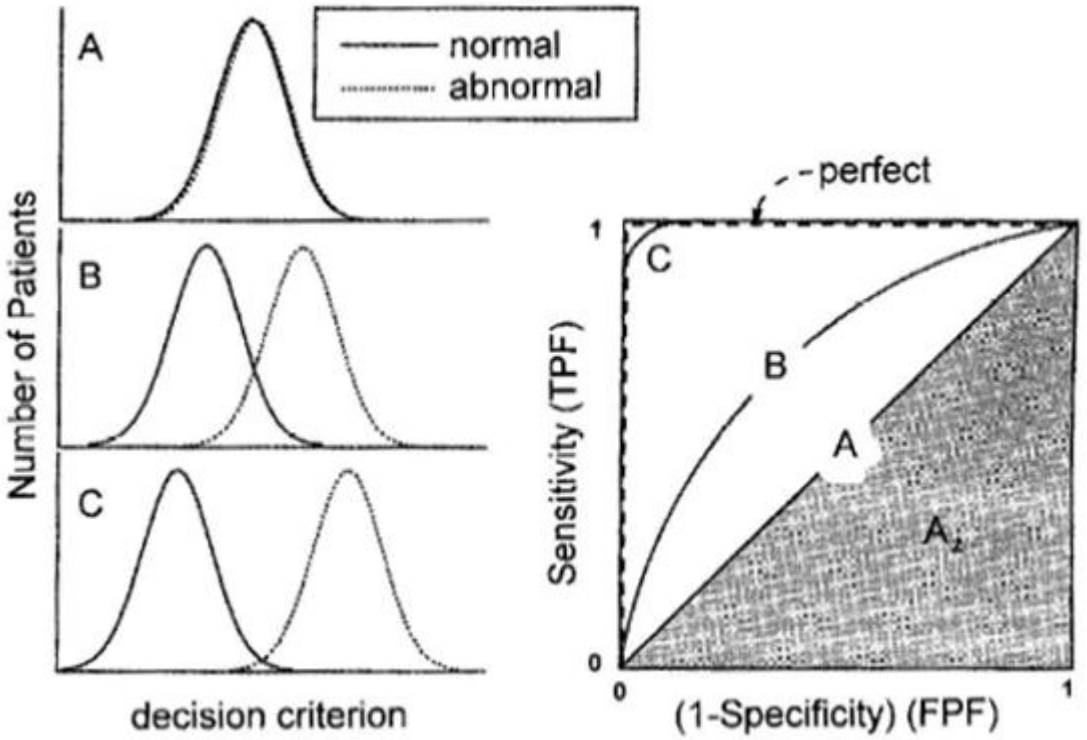
Classifier A can't distinguish between normal and abnormal.

B is better but makes some mistakes.

C makes very few mistakes.



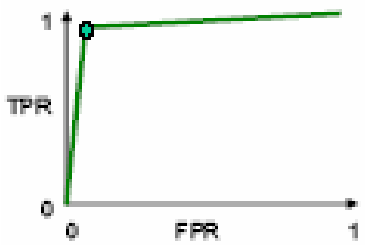
"Perfect" means no false positives and no false negatives.



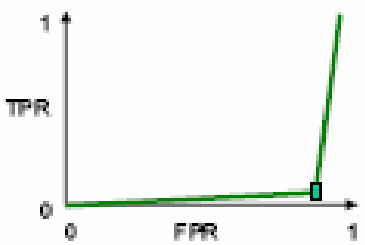
ROC analysis

- ROC = receiver operator/operating characteristic/curve

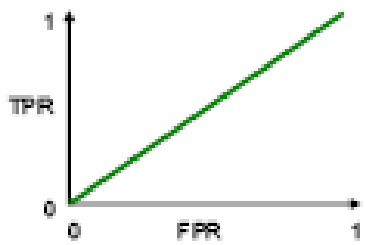
- ROC space: good and bad classifiers.



- Good classifier.
  - High TPR.
  - Low FPR.

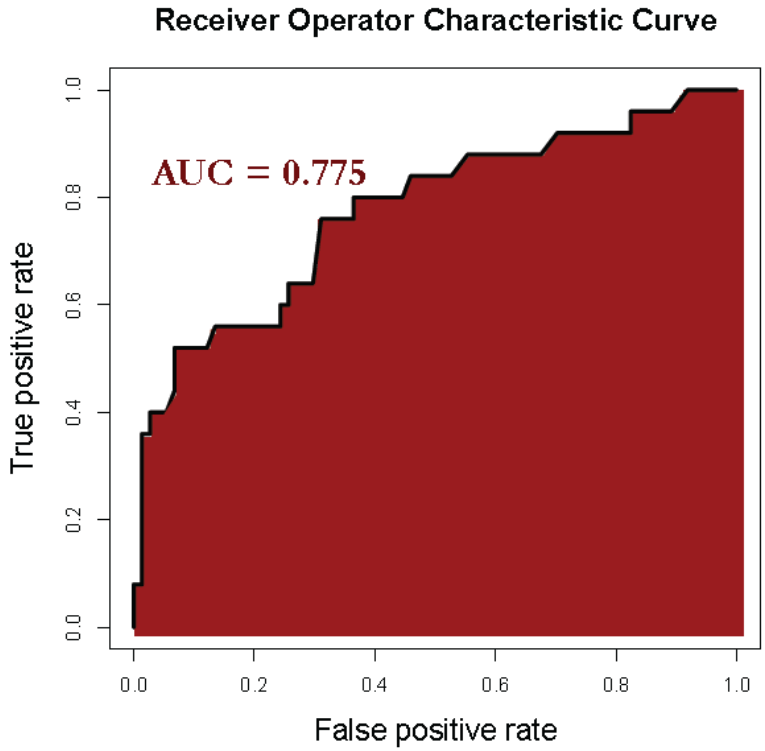


- Bad classifier.
  - Low TPR.
  - High FPR.



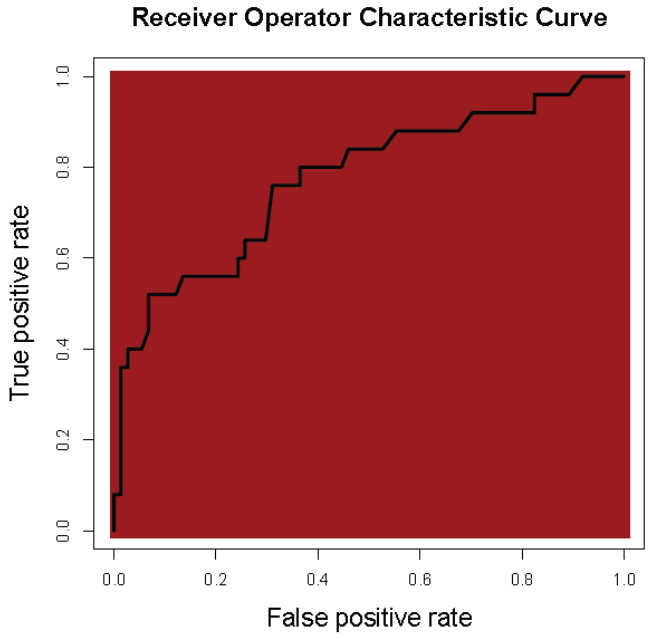
- Bad classifier (real picture).

Area Under the ROC Curve (AUC)



Area Under the ROC Curve (AUC)

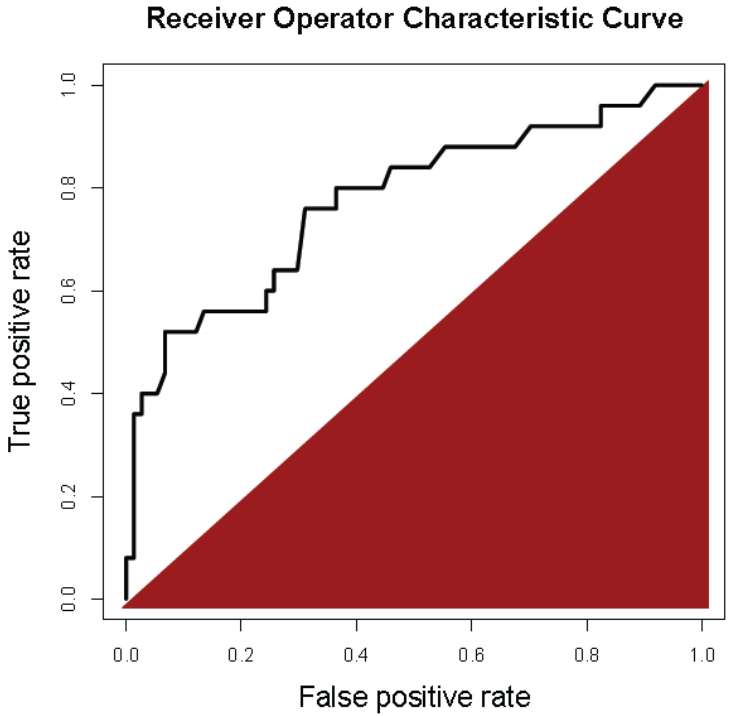
- What is a good AUC?
  - Maximum of 1 (perfect prediction)



Area Under the ROC Curve (AUC)

- What is a good AUC?
- Maximum of 1 (perfect prediction)
- Minimum of 0.5

  (just guessing)

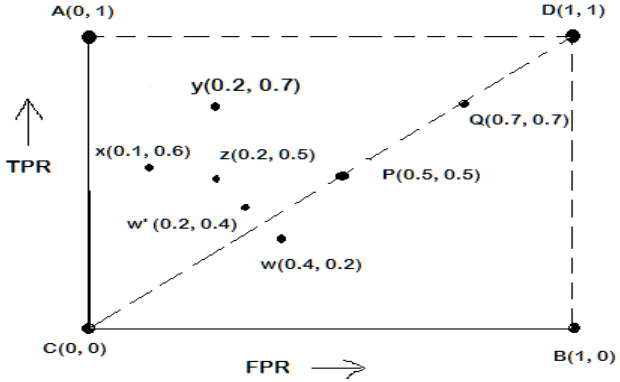


Selecting a Threshold using ROC

- Choose best threshold for best trade off
  - cost of failing to detect positives
  - costs of raising false alarms

ROC Plot

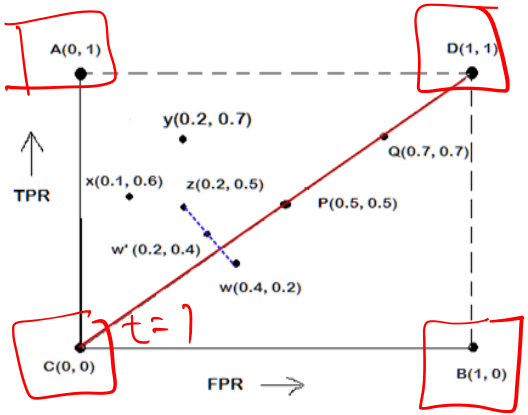
- A typical look of ROC plot with few points in it is shown in the following figure.



- Note the four cornered points are the four extreme cases of classifiers

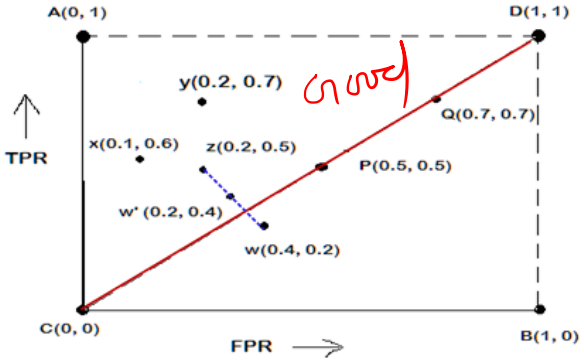
Interpretation of Different Points in ROC Plot

- The four points (A, B, C, and D)
- A: TPR = 1, FPR = 0, the ideal model, i.e., the perfect classifier, no false results
- B: TPR = 0, FPR = 1, the worst classifier, not able to predict a single instance
- C: TPR = 0, FPR = 0, the model predicts every instance to be a Negative class, i.e., it is an ultra-conservative classifier
- D: TPR = 1, FPR = 1, the model predicts every instance to be a Positive class, i.e., it is an ultra-liberal classifier



Interpretation of Different Points in ROC Plot

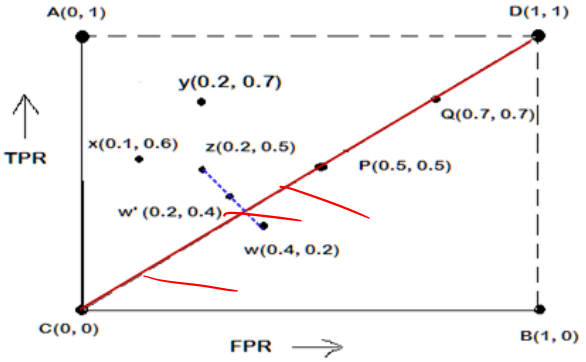
- Let us interpret the different points in the ROC plot.
- **The points on the upper diagonal region**
- All points, which reside on upper-diagonal region are corresponding to classifiers “good” as their TPR is as good as FPR (i.e., FPRs are lower than TPRs)
- Here, X is better than Z as X has higher TPR and lower FPR than Z.
- If we compare X and Y, neither classifier is superior to the other



Interpretation of Different Points in ROC Plot

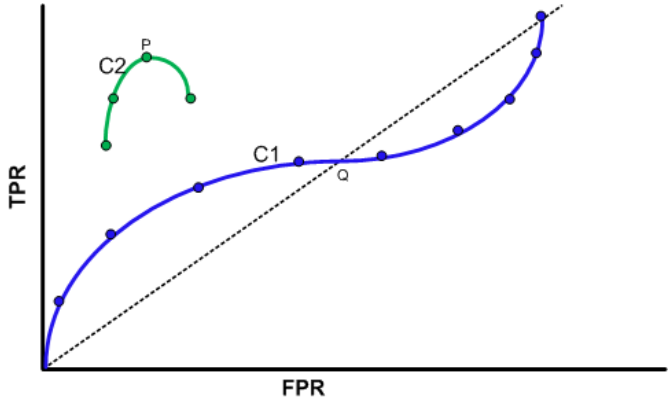
- Let us interpret the different points in the ROC plot.
- The points on the lower diagonal region
  - The Lower-diagonal triangle corresponds to the classifiers that are worst than random classifiers
  - A classifier that is worser than random guessing, simply by reversing its prediction, we can get good results.

W'(0.2, 0.4) is the better version than W(0.4, 0.2), W' is a mirror reflection of W



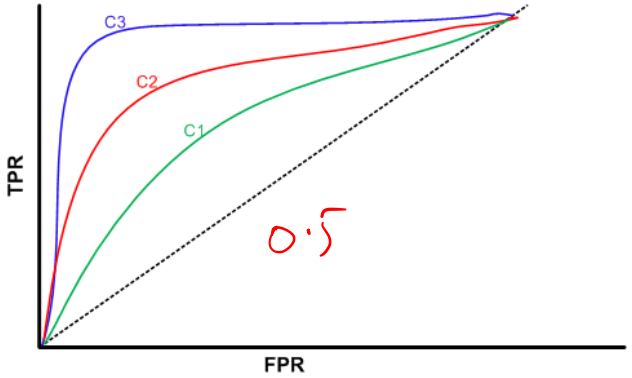
Tuning a Classifier through ROC Plot

- Using ROC plot, we can compare two or more classifiers by their TPR and FPR values and this plot also depicts the trade-off between TPR and FPR of a classifier.
- Examining ROC curves can give insights into the best way of tuning parameters of classifier.
- For example, in the curve C2, the result is degraded after the point P.
- Similarly for the observation C1, beyond Q the settings are not acceptable.

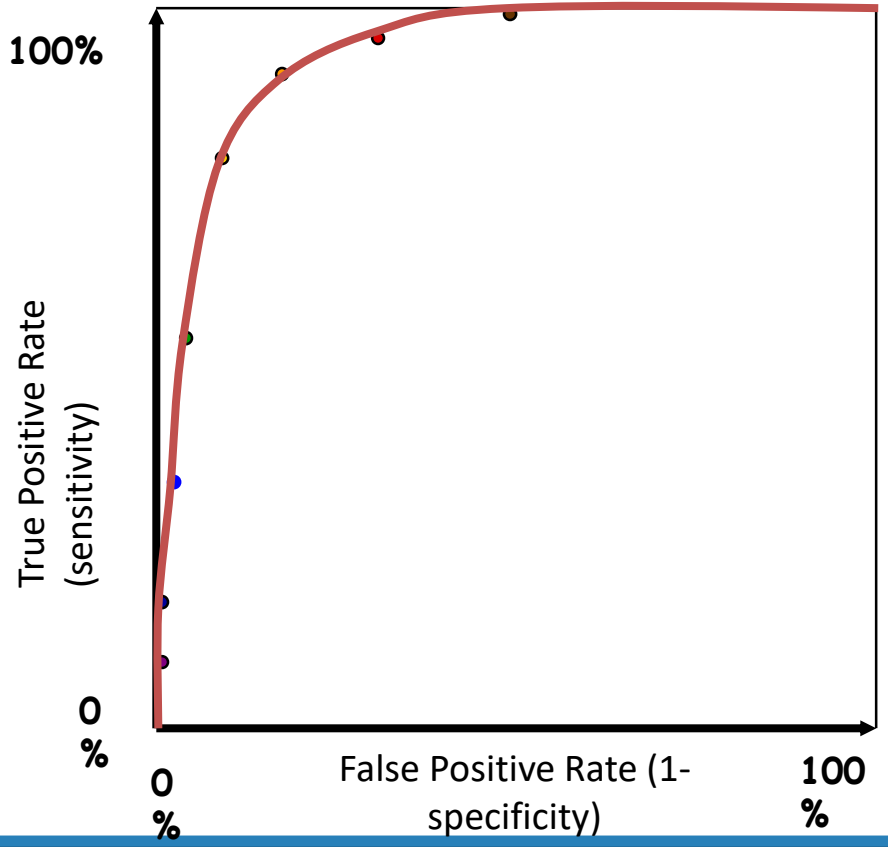


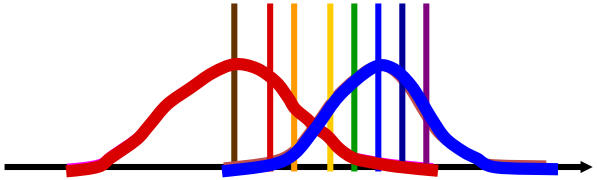
Comparing Classifiers trough ROC Plot

- We can use the concept of "area under curve" (AUC) as a better method to compare two or more classifiers.
- If a model is perfect, then its AUC = 1.
- If a model simply performs random guessing, then its AUC = 0.5
- A model that is strictly better than other, would have a larger value of AUC than the other.
- Here, C3 is best, and C2 is better than C1 as AUC(C3)>AUC(C2)>AUC(C1).



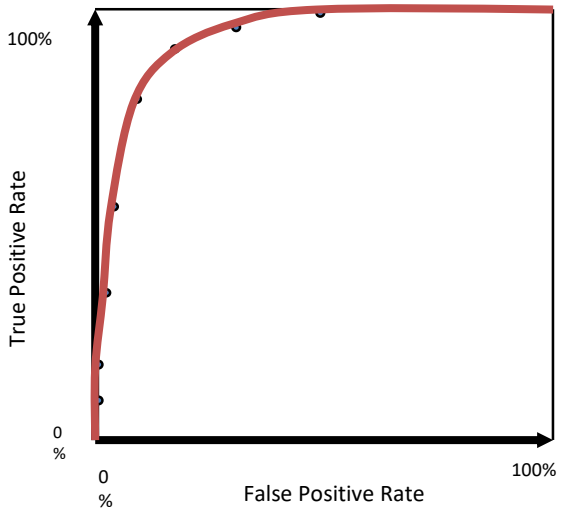
ROC curve


100%
True Positive Rate (sensitivity)
0 %
0 %
False Positive Rate (1-specificity)
100 %

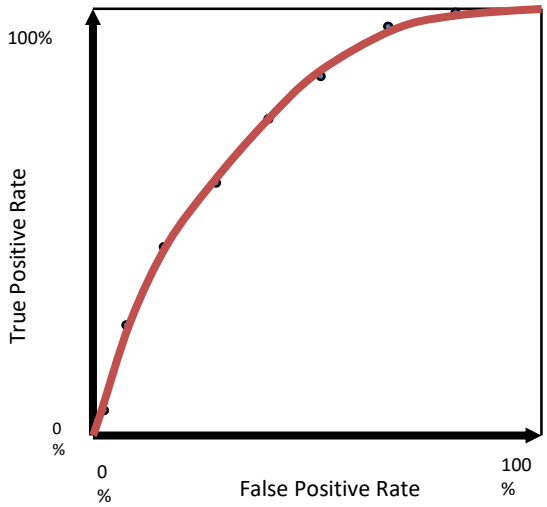


ROC curve comparison

A good test:

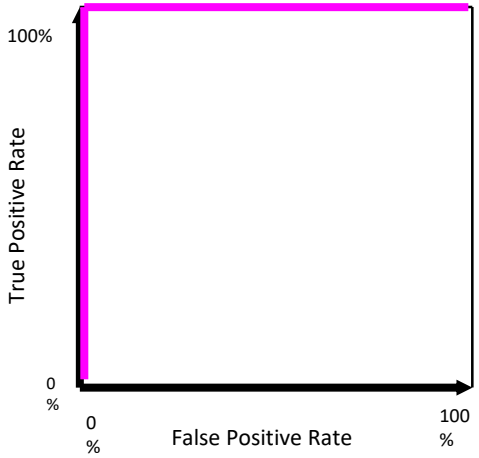


A poor test:



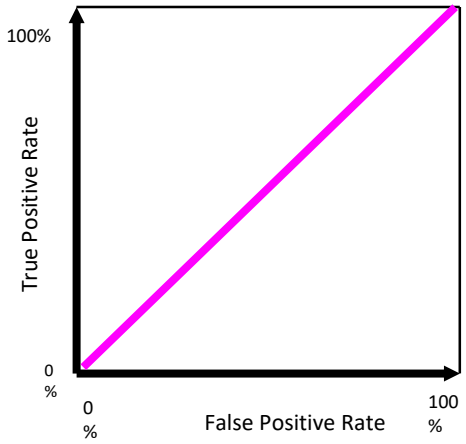
ROC curve extremes

Best Test:



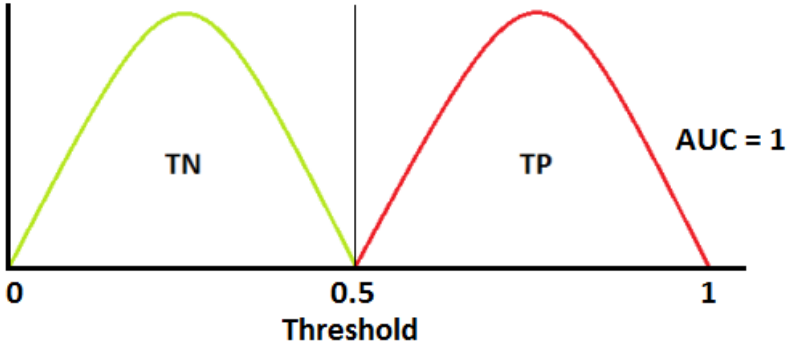
The distributions don't overlap at all

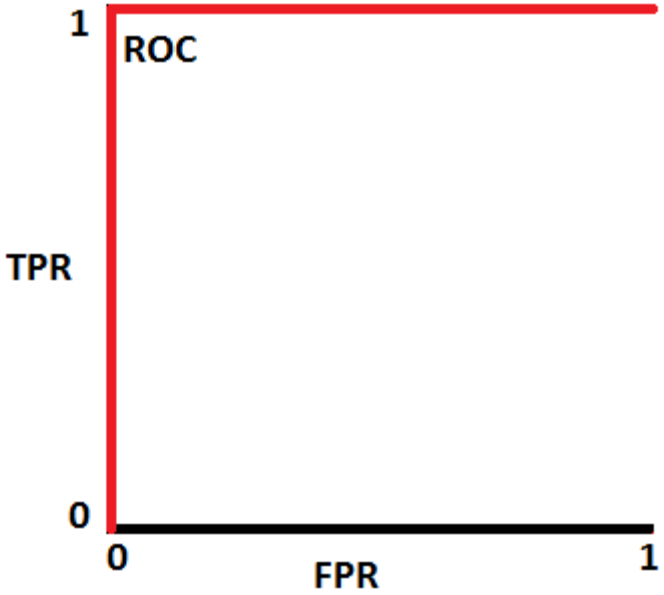
Worst test:



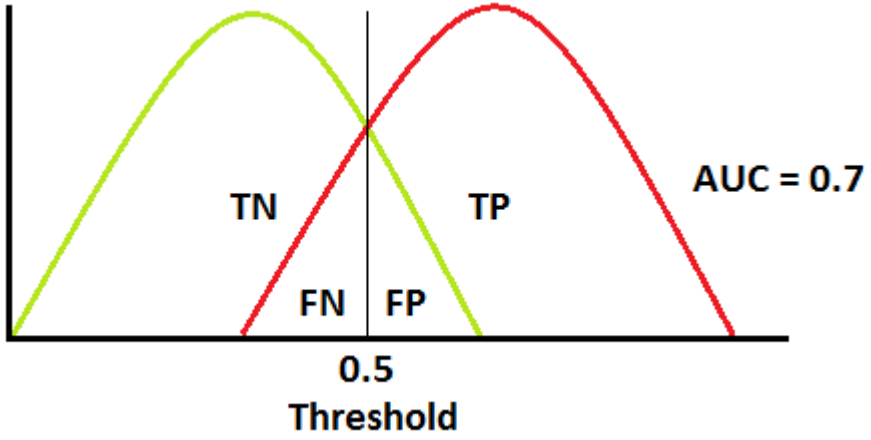
The distributions overlap completely

ROC curve extremes

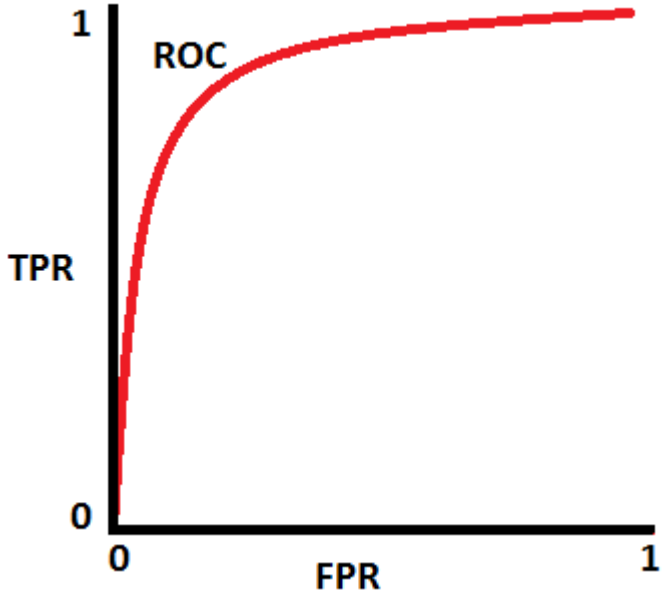




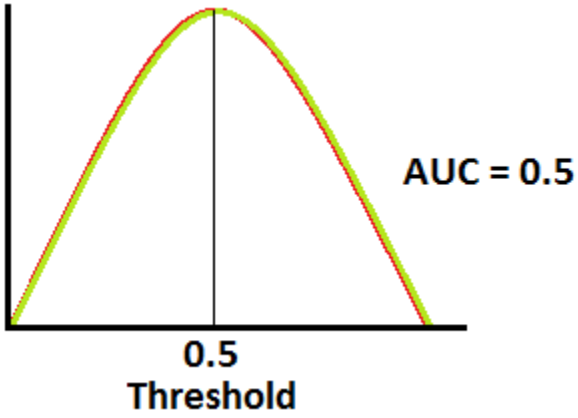
Typical ROC

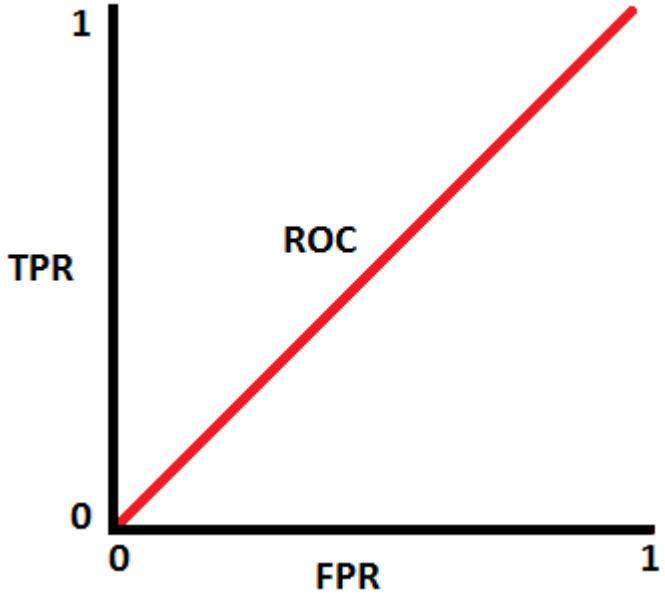


AUC = 0.7



ROC curve extremes





Example

- Let us consider an application of logistic regression involving a direct mail promotion being used by Simmons Stores.
- Simmons owns and operates a national chain of women's apparel stores.
- Five thousand copies of an expensive four-color sales catalog have been printed, and each catalog includes a coupon that provides a $50 discount on purchases of $200 or more.
- The catalogs are expensive and Simmons would like to send them to only those customers who have the highest probability of using the coupon.

Sources: Statistics for Business and Economics,11th Edition by David R. Anderson (Author), Dennis J. Sweeney (Author), Thomas A. Williams (Author)

Variables

- Management thinks that annual spending at Simmons Stores and whether a customer has a Simmons credit card are two variables that might be helpful in predicting whether a customer who receives the catalog will use the coupon.
- Simmons conducted a pilot study using a random sample of 50 Simmons credit card customers and 50 other customers who do not have a Simmons credit card.
- Simmons sent the catalog to each of the 100 customers selected.
- At the end of a test period, Simmons noted whether the customer used the coupon or not?

Data (10 customer out of 100)

Customer	Spending	Card	Coupon
1	2.291	1	0
2	3.215	1	0
3	2.135	1	0
4	3.924	0	0
5	2.528	1	0
6	2.473	0	1
7	2.384	0	0
8	7.076	0	0
9	1.182	1	1
10	3.345	0	0

Explanation of Variables

- The amount each customer spent last year at Simmons is shown in thousands of dollars and the credit card information has been coded as 1 if the customer has a Simmons credit card and 0 if not.
- In the Coupon column, a 1 is recorded if the sampled customer used the coupon and 0 if not.

Loading data file and get some statistical detail

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
```

In [2]:
```
data = pd.read_excel('Simmons.xls')
data.head()
```

Out[2]:

	Customer	Spending	Card	Coupon
0	1	2.291	1	0
1	2	3.215	1	0
2	3	2.135	1	0
3	4	3.924	0	0
4	5	2.528	1	0

In [3]:
```
data.describe() #it is used to get some statistical detail
```

Out[3]:

	Customer	Spending	Card	Coupon
count	100.000000	100.000000	100.000000	100.000000
mean	50.500000	3.333790	0.500000	0.400000
std	29.011492	1.741298	0.502519	0.492366
min	1.000000	1.058000	0.000000	0.000000
25%	25.750000	2.059000	0.000000	0.000000
50%	50.500000	2.805500	0.500000	0.000000
75%	75.250000	4.468250	1.000000	1.000000
max	100.000000	7.076000	1.000000	1.000000

Method's description

- Dataframe.describe(): This method is used to get basic statistical details such as central tendency, dispersion and shape of dataset's distribution.
- Numpy.unique(): This method gives unique values in particular column.
- Series.value_counts(): Returns object containing counts of unique values.
- ravel(): It will return one dimensional array with all the input array elements.

Split dataset into training and testing sets

In [4]:
```
data['Coupon'].unique() # It gives unique value in perticular column
```

Out[4]:
```
array([0, 1], dtype=int64)
```

In [5]:
```
data['Coupon'].value_counts()
```

Out[5]:
```
0    60
1    40
Name: Coupon, dtype: int64
```

In [7]:
```
from sklearn import linear_model
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

In [8]:
```
x = data[['Card','Spending']]
y = data['Coupon'].values.reshape(-1,1)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.25, random_state = 42)
```

In [9]:
```
len(x_train), len(y_train), len(x_test), len(y_test)
```

Out[9]:
```
(75, 75, 25, 25)
```

Building the model and predicting values

In [10]:
```
Lreg = LogisticRegression(solver='lbfgs')
Lreg.fit(x_train, y_train.ravel()) #ravel() will return 1D array with all the input-array elements
```

Out[10]:
```
LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
          intercept_scaling=1, max_iter=100, multi_class='warn',
          n_jobs=None, penalty='l2', random_state=None, solver='lbfgs',
          tol=0.0001, verbose=0, warm_start=False)
```

In [11]:
```
y_predict = Lreg.predict(x_test)
y_predict
```

Out[11]:
```
array([1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,
       1, 0, 0], dtype=int64)
```

In [12]:
```
y_predict_train = Lreg.predict(x_train)
y_predict_train
```

Out[12]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 1, 0, 0, 0, 0, 1, 1, 0], dtype=int64)
```

Calculate probability of predicting data values

In [13]:
```
1 y_prob_train = Lreg.predict_proba(x_train)[:,1]
2 y_prob_train.reshape(1,-1)
```

Out[13]:
```
array([[0.49622117, 0.32880793, 0.44329114, 0.33320924, 0.41456465,
        0.32890329, 0.3975043 , 0.66921229, 0.25844531, 0.63672372,
        0.29274386, 0.28466974, 0.5159296 , 0.41992276, 0.24342356,
        0.528514  , 0.47965107, 0.52805789, 0.33191449, 0.27457435,
        0.49179296, 0.63261616, 0.24690181, 0.47089452, 0.27842076,
        0.41663875, 0.36155602, 0.49970327, 0.23621636, 0.37860052,
        0.48809323, 0.28877877, 0.28563859, 0.37231882, 0.65309742,
        0.43807264, 0.33638478, 0.40406607, 0.23431177, 0.37282384,
        0.49970327, 0.39768396, 0.32880793, 0.25782472, 0.47393834,
        0.42878861, 0.26520939, 0.33320924, 0.54682499, 0.45446086,
        0.44326597, 0.4965167 , 0.60065954, 0.38989654, 0.49149447,
        0.27414424, 0.27785686, 0.67464141, 0.28195004, 0.48593427,
        0.38633222, 0.31373499, 0.42810085, 0.27418723, 0.44371771,
        0.41629601, 0.642004  , 0.6571001 , 0.44068025, 0.28195004,
        0.40217015, 0.43807264, 0.50977653, 0.57944626, 0.2904233 ]])
```

75

In [14]:
```
1 y_prob = Lreg.predict_proba(x_test)[:,1]
2 y_prob.reshape(1,-1)
3 y_prob
```

Out[14]:
```
array([0.52802946, 0.49516653, 0.45703306, 0.27712052, 0.34390047,
       0.26825171, 0.27712052, 0.607686  , 0.42836534, 0.43637155,
       0.31387455, 0.23676248, 0.45703306, 0.43602768, 0.37596116,
       0.44900317, 0.46952365, 0.68521935, 0.25167254, 0.47073304,
       0.42361093, 0.56580644, 0.52792177, 0.40302605, 0.27457435])
```

25

Summary for logistic model

In [15]:

```
x = data[['Spending', 'Card']]
y = data['Coupon']

import statsmodels.api as sm
x1 = sm.add_constant(x)
logit_model=sm.Logit(y,x1)
result=logit_model.fit()
print(result.summary())
```

```
Optimization terminated successfully.
         Current function value: 0.604869
         Iterations 5
                           Logit Regression Results
==============================================================================
Dep. Variable:                 Coupon   No. Observations:                  100
Model:                          Logit   Df Residuals:                       97
Method:                           MLE   Df Model:                            2
Date:                Mon, 16 Sep 2019   Pseudo R-squ.:                  0.1012
Time:                        10:15:13   Log-Likelihood:                -60.487
converged:                       True   LL-Null:                       -67.301
                                        LLR p-value:                  0.001098
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const         -2.1464      0.577     -3.718      0.000      -3.278      -1.015
Spending       0.3416      0.129      2.655      0.008       0.089       0.594
Card           1.0987      0.445      2.471      0.013       0.227       1.970
==============================================================================
```

Accuracy Checking

- By using accuracy_score function.
- By using confusion matrix

	Predicted (0)	Predicted (1)
Actual (0)	True Negative(tn)	False Positive(fp)
Actual (1)	False Negative(fn)	True Positive(tp)

Calculating Accuracy Score using Confusion Matrix

In [16]:
```
1 from sklearn.metrics import accuracy_score
2 score =accuracy_score(y_test,y_predict)
3 score
```

Out[16]: 0.76

In [17]:
```
1 from sklearn.metrics import confusion_matrix
2 confusion_matrix(y_test, y_predict)
```

Out[17]:
```
array([[15,  1],
       [ 5,  4]], dtype=int64)
```

In [18]:
```
1 tn, fp, fn, tp = confusion_matrix(y_test, y_predict).ravel()
2 print("True Negatives: ",tn)
3 print("False Positives: ",fp)
4 print("False Negatives: ",fn)
5 print("True Positives: ",tp)
```

```
True Negatives:  15
False Positives:  1
False Negatives:  5
True Positives:  4
```

Generating Classification Report

```
In [19]:
1 from sklearn.metrics import classification_report
2 print(classification_report(y_test, y_predict))
```

	precision	recall	f1-score	support
0	0.75	0.94	0.83	16
1	0.80	0.44	0.57	9
micro avg	0.76	0.76	0.76	25
macro avg	0.78	0.69	0.70	25
weighted avg	0.77	0.76	0.74	25

- Recall gives us an idea about when it's actually yes, how often does it predict yes.
- Precision tells us about when it predicts yes, how often is it correct

Interpreting Classification Report

- Precision = tp / (tp + fp)
- Accuracy = (tp + tn) / (tp + tn + fp + fn)
- Recall= tp / (tp + fn)

	Predicted (0)	Predicted (1)
Actual (0)	tn	fp
Actual (1)	fn	tp

Thank You

IIT ROORKEE

FREE ONLINE EDUCATION swayam शिक्षित भारत, उन्नत भारत

NPTEL ONLINE CERTIFICATION COURSE

Performance of Logistic Model-III

Dr A. RAMESH

DEPARTMENT OF MANAGEMENT STUDIES

Agenda

Python demo for accuracy prediction in logistic regression model using Receiver operating characteristics curve

Sensitivity and Specificity

- For checking, what type of error we are making; we use two parameters-

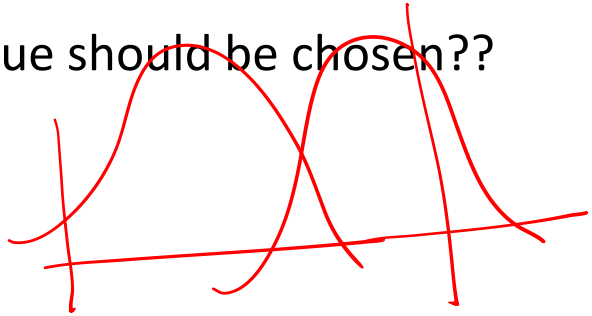
1. Sensitivity = tp/(tp+fn) ⟶ True Positive Rate(tpr)

2. Specificity = tn/(tn+fp) ⟶ True Negative Rate (tnr)

Specificity and Sensitivity Relationship with Threshold

Threshold (Lower)	Sensitivity (↑)	Specificity (↓)
Threshold (Higher)	Sensitivity (↓)	Specificity (↑)

Which threshold value should be chosen??



Measuring Accuracy, Specificity and Sensitivity

In [20]:
```
Accuracy = (tp + tn) / (tp + tn + fp + fn)
print("Accuracy {:0.2f}".format(Accuracy))
```

```
Accuracy 0.76
```

In [21]:
```
Specificity = tn/(tn+fp)
print("Specificity {:0.2f}".format(Specificity))
```

```
Specificity 0.94
```

In [22]:
```
Sensitivity = tp/(tp+fn)
print("Sensitivity {:0.2f}".format(Sensitivity))
```

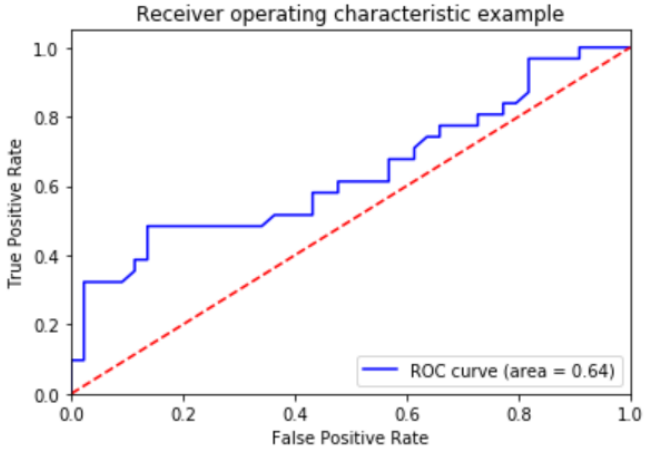
```
Sensitivity 0.44
```

$t = 0.5$

ROC Curve for Training dataset

In [23]:
```
from sklearn.metrics import roc_auc_score
from sklearn.metrics import roc_curve, auc
log_ROC_AUC1 = roc_auc_score(y_train, y_predict_train)
fpr1, tpr1, thresholds1= roc_curve(y_train, y_prob_train)
roc_auc1 = auc(fpr1, tpr1)
```

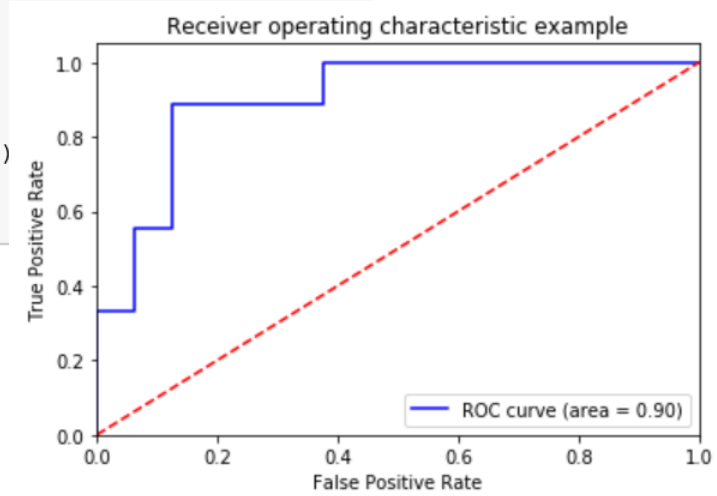
In [24]:
```
plt.figure()
plt.plot(fpr1, tpr1, color='blue', label='ROC curve (area = %0.2f)' % roc_auc1)
plt.plot([0, 1], [0, 1],'r--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic example')
plt.legend(loc="lower right")
plt.show()
```



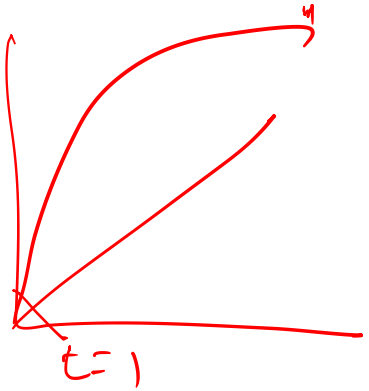
ROC Curve for Test data set

In [25]:
```
log_ROC_AUC = roc_auc_score(y_test, y_predict)
fpr, tpr, thresholds= roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)
```

In [26]:
```
plt.figure()
plt.plot(fpr, tpr, color='blue', label='ROC curve (area = %0.2f)' % roc_auc)
plt.plot([0, 1], [0, 1],'r--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic example')
plt.legend(loc="lower right")
plt.show()
```



Threshold value selection



- The outcome of logistic regression model is a probability.
- Selecting a good threshold value is often challenging.
- Threshold values on ROC curve –

Threshold = 1	TPR = 0	FPR = 0
Threshold = 0	TPR = 1	FPR = 1

- Threshold values are often selected based on which errors are bettor.

Accuracy checking for different threshold values

In [27]:
```
from sklearn.preprocessing import binarize
y_predict_class1 = binarize(y_prob.reshape(1,-1), 0.35)[0]
y_predict_class1
```

Out[27]:
```
array([1., 1., 1., 0., 0., 0., 0., 1., 1., 1., 0., 0., 1., 1., 1., 1., 1.,
       1., 0., 1., 1., 1., 1., 1., 0.])
```

In [28]:
```
#converting the array from float data type to integer data type
y_predict_class1 = y_predict_class1.astype(int)
y_predict_class1
```

Out[28]:
```
array([1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1,
       1, 1, 0])
```

In [29]:
```
confusion_matrix_2 = confusion_matrix(y_test, y_predict_class1)
print(confusion_matrix_2)
```

```
[[8 8]
 [0 9]]
```

Accuracy checking for different threshold values

In [30]:

```
tn, fp, fn, tp = confusion_matrix(y_test, y_predict_class1).ravel()
print("True Negatives: ",tn)
print("False Positives: ",fp)
print("False Negatives: ",fn)
print("True Positives: ",tp)
```

```
True Negatives:  8
False Positives:  8
False Negatives:  0
True Positives:  9
```

In [31]:

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_predict_class1))
```

	precision	recall	f1-score	support
0	1.00	0.50	0.67	16
1	0.53	1.00	0.69	9
micro avg	0.68	0.68	0.68	25
macro avg	0.76	0.75	0.68	25
weighted avg	0.83	0.68	0.68	25

Accuracy checking for different threshold values

In [32]:
```
from sklearn.preprocessing import binarize
y_predict_class2 = binarize(y_prob.reshape(1,-1), 0.50)[0]
y_predict_class2
```

Out[32]:
```
array([1., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
       1., 0., 0., 0., 1., 1., 0., 0.])
```

In [33]:
```
confusion_matrix_3 = confusion_matrix(y_test, y_predict_class2)
print(confusion_matrix_3)
```

```
[[15  1]
 [ 5  4]]
```

In [34]:
```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_predict_class2))
```

```
              precision    recall  f1-score   support

           0       0.75      0.94      0.83        16
           1       0.80      0.44      0.57         9

   micro avg       0.76      0.76      0.76        25
   macro avg       0.78      0.69      0.70        25
weighted avg       0.77      0.76      0.74        25
```

Accuracy checking for different threshold values

In [35]:
```
from sklearn.preprocessing import binarize
y_predict_class3 = binarize(y_prob.reshape(1,-1), 0.70)[0]
y_predict_class3
```

Out[35]:
```
array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
       0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

In [36]:
```
confusion_matrix_4 = confusion_matrix(y_test, y_predict_class3)
print(confusion_matrix_4)
```

```
[[16  0]
 [ 9  0]]
```

In [37]:
```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_predict_class3))
```

```
              precision    recall  f1-score   support

           0       0.64      1.00      0.78        16
           1       0.00      0.00      0.00         9

   micro avg       0.64      0.64      0.64        25
   macro avg       0.32      0.50      0.39        25
weighted avg       0.41      0.64      0.50        25
```

Calculating Optimal Threshold Value

In [38]:
```
from sklearn.metrics import roc_curve, auc
```

In [39]:
```
fpr, tpr, thresholds= roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)
```

In [40]:
```
print("Area under the ROC curve : %f" % roc_auc)
```

In [41]:
```
import numpy as np
i = np.arange(len(tpr)) # index for df
roc = pd.DataFrame({'fpr' : pd.Series(fpr, index=i),'tpr' : pd.Series(tpr, index = i),
                    '1-fpr' : pd.Series(1-fpr, index = i), 'tf' : pd.Series(tpr - (1-fpr), index = i),
                    'thresholds' : pd.Series(thresholds, index = i)})
roc.iloc[(roc.tf-0).abs().argsort()[:1]]
```

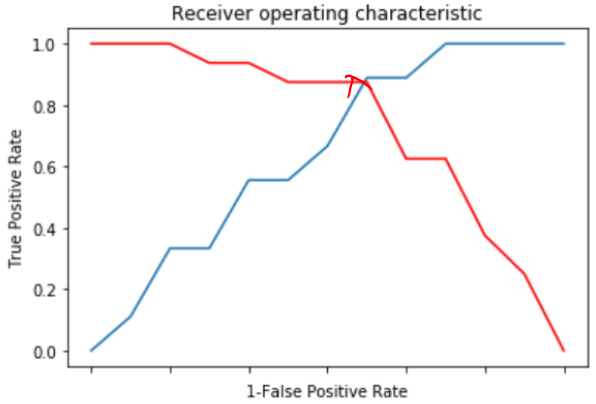
Out[41]:

	fpr	tpr	1-fpr	tf	thresholds
7	0.125	0.888889	0.875	0.013889	0.457033

Optimal Threshold Value in ROC Curve

In [42]:
```
1  fig, ax = plt.subplots()
2  plt.plot(roc['tpr'])
3  plt.plot(roc['1-fpr'], color = 'red')
4  plt.xlabel('1-False Positive Rate')
5  plt.ylabel('True Positive Rate')
6  plt.title('Receiver operating characteristic')
7  ax.set_xticklabels([])
```

Out[42]: []



Classification Report using Optimal Threshold Value

In [43]:
```
from sklearn.preprocessing import binarize
y_predict_class4 = binarize(y_prob.reshape(1,-1), 0.45)[0]
y_predict_class4
```

Out[43]:
```
array([1., 1., 1., 0., 0., 0., 0., 1., 0., 0., 0., 0., 1., 0., 0., 0., 1.,
       1., 0., 1., 0., 1., 1., 0., 0.])
```

In [44]:
```
confusion_matrix_5 = confusion_matrix(y_test, y_predict_class4)
print(confusion_matrix_5)
```

```
[[14  2]
 [ 1  8]]
```

In [45]:
```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_predict_class4))
```

```
              precision    recall  f1-score   support

           0       0.93      0.88      0.90        16
           1       0.80      0.89      0.84         9

   micro avg       0.88      0.88      0.88        25
   macro avg       0.87      0.88      0.87        25
weighted avg       0.89      0.88      0.88        25
```

Thank You

IIT ROORKEE

FREE ONLINE EDUCATION swayam शिक्षित भारत, उन्नत भारत

NPTEL ONLINE CERTIFICATION COURSE

Regression Analysis Model Building - I

Dr. A. Ramesh

DEPARTMENT OF MANAGEMENT STUDIES

Introduction

- Model building is the process of developing an estimated regression equation that describes the relationship between a dependent variable and one or more independent variables.
- The major issues in model building are finding the proper functional form of the relationship and selecting the independent variables to be included in the model.

General Linear Regression Model

- Suppose we collected data for one dependent variable y and k independent variables $x_1, x_2, \ldots, x_k$.
- Objective is to use these data to develop an estimated regression equation that provides the best relationship between the dependent and independent variables.

GENERAL LINEAR MODEL

$$y = \beta_0 + \beta_1 z_1 + \beta_2 z_2 + \cdots + \beta_p z_p + \epsilon$$

- z_j (where j =1, 2, . . . , p) is a function of $x_1, x_2, \ldots, x_k$ (the variables for which data are collected).
- In some cases, each z_j may be a function of only one x variable.

Simple first-order model with one predictor variable

$$y = \beta_0 + \beta_1 x_1 + \epsilon$$

Modelling Curvilinear Relationships

- To illustrate, let us consider the problem facing Reynolds, Inc., a manufacturer of industrial scales and laboratory equipment.
- Managers at Reynolds want to investigate the relationship between length of employment of their salespeople and the number of electronic laboratory scales sold.
- Table in the next slide gives the number of scales sold by 15 randomly selected salespeople for the most recent sales period and the number of months each salesperson has been employed by the firm.

Sources: Statistics for Business and Economics,11th Edition by David R. Anderson (Author), Dennis J. Sweeney (Author), Thomas A. Williams (Author)

Data

Scales Sold	Months Employed
275	41
296	106
317	76
376	104
162	22
150	12
367	85
308	111
189	40
235	51
83	9
112	12
67	6
325	56
189	19

Importing libraries and table

```
In [1]: import pandas as pd
        import numpy as np
        import matplotlib.pyplot as plt
        import statsmodels.api as sm
```

```
In [9]: tbl1 = pd.read_excel('Reynolds.xlsx')
        tbl1
```

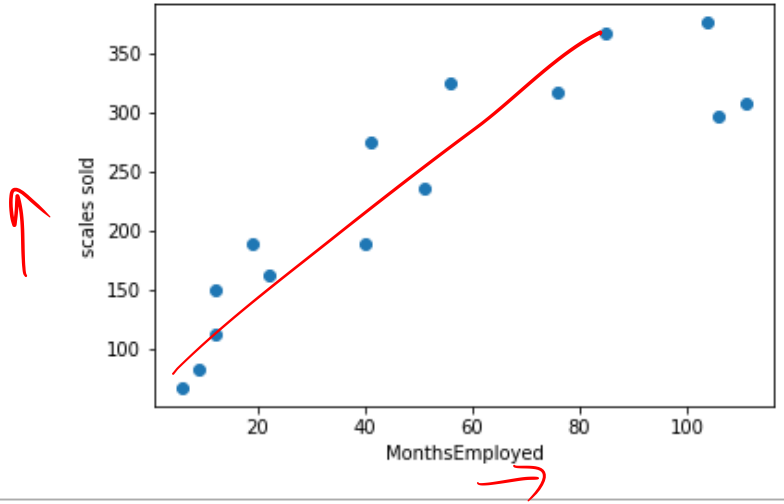
Out[9]:

	ScalesSold	MonthsEmployed
0	275	41
1	296	106
2	317	76
3	376	104
4	162	22
5	150	12
6	367	85
7	308	111
8	189	40
9	235	51
10	83	9
11	112	12
12	67	6

SCATTER DIAGRAM FOR THE REYNOLDS EXAMPLE

```
In [13]: plt.scatter(tbl1['MonthsEmployed'],tbl1['ScalesSold'])
         plt.ylabel('scales sold')
         plt.xlabel('MonthsEmployed')
```

```
Out[13]: Text(0.5,0,'MonthsEmployed')
```



Python code for the Reynolds example: first-order model

```
In [14]: x =tbl1['MonthsEmployed']
         y = tbl1['ScalesSold']
         x2 = sm.add_constant(x)
         model = sm.OLS(y,x2)
         Model = model.fit()
         print(Model.summary())
```

```
C:\Users\Somi\Anaconda3\lib\site-packages\scipy\stats\stats.py:1394: UserWarning: kurtosistest c
ing anyway, n=15
  "anyway, n=%i" % int(n))
```

OLS Regression Results

Dep. Variable:	ScalesSold	R-squared:	0.781
Model:	OLS	Adj. R-squared:	0.764
Method:	Least Squares	F-statistic:	46.41
Date:	Thu, 12 Sep 2019	Prob (F-statistic):	1.24e-05
Time:	12:15:26	Log-Likelihood:	-78.745
No. Observations:	15	AIC:	161.5
Df Residuals:	13	BIC:	162.9
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	111.2279	21.628	5.143	0.000	64.503	157.952
MonthsEmployed	2.3768	0.349	6.812	0.000	1.623	3.131

Omnibus:	1.043	Durbin-Watson:	2.261
Prob(Omnibus):	0.594	Jarque-Bera (JB):	0.723
Skew:	0.052	Prob(JB):	0.697
Kurtosis:	1.930	Cond. No.	105.

$y = 111.22 + 2.37$ Months Employed

First-order regression equation

$$\text{Sales} = 111 + 2.38\,\text{Months}$$

where

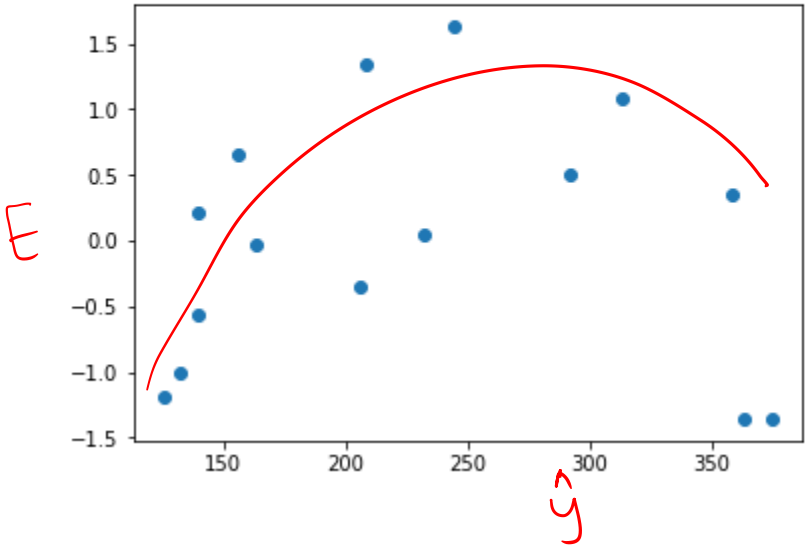
Sales = number of electronic laboratory scales sold
Months = the number of months the salesperson has been employed

Standardized residual plot for the Reynolds example: first-order model

```
In [18]: E=Model.resid_pearson

In [19]: E

Out[19]: array([ 1.33945744, -1.35645713,  0.50765989,  0.35518943, -0.03063607,
                 0.20702037,  1.08543558, -1.35411191, -0.34936157,  0.05163116,
                -1.00208207, -0.56041143, -1.18121025,  1.62923113,  0.65864542])

In [42]: yhat = Model.predict(x2)
         yhat

Out[42]: 0     208.675693
         1     363.166061
         2     291.862814
         3     358.412511
         4     163.516970
         5     139.749221
         6     313.253788
         7     375.049935
         8     206.298918
         9     232.443442
         10    132.618896
         11    139.749221
         12    125.488571
         13    244.327316
         14    156.386645
         dtype: float64
```

Standardized residual plot for the Reynolds example: first-order model

```
In [25]: plt.scatter(yhat,E)
Out[25]: <matplotlib.collections.PathCollection at 0x16096243b38>
```



Need for curvilinear relationship

- Although the computer output shows that the relationship is significant (*p*-value .000) and that a linear relationship explains a high percentage of the variability in sales (R-sq 78.1%), the standardized residual plot suggests that a curvilinear relationship is needed.

Second-order model with one predictor variable

- Set $Z_1 = x_1$ and $Z_2 = X^2$

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_1^2 + \epsilon$$

New Data set

- *The data for the* MonthsSq *independent variable is obtained by squaring the values of* Months.

```
In [29]: X_sq = (x**2)
         X_sq

Out[29]: 0      1681
         1     11236
         2      5776
         3     10816
         4       484
         5       144
         6      7225
         7     12321
         8      1600
         9      2601
         10       81
         11      144
         12       36
         13     3136
         14      361
         Name: MonthsEmployed, dtype: int64
```

Python output for the Reynolds example: second-order model

```
In [31]: x_new =np.column_stack((x,X_sq))
         x_new2 = sm.add_constant(x_new)
         model2 = sm.OLS(y,x_new2)
         Model2 = model2.fit()
         print(Model2.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:             ScalesSold   R-squared:                       0.902
Model:                            OLS   Adj. R-squared:                  0.886
Method:                 Least Squares   F-statistic:                     55.36
Date:                Thu, 12 Sep 2019   Prob (F-statistic):           8.75e-07
Time:                        12:38:01   Log-Likelihood:                -72.704
No. Observations:                  15   AIC:                             151.4
Df Residuals:                      12   BIC:                             153.5
Df Model:                           2
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         45.3476     22.775      1.991      0.070      -4.274      94.969
x1             6.3448      1.058      5.998      0.000       4.040       8.650
x2            -0.0345      0.009     -3.854      0.002      -0.054      -0.015
==============================================================================
Omnibus:                        2.162   Durbin-Watson:                   1.313
Prob(Omnibus):                  0.339   Jarque-Bera (JB):                1.003
Skew:                          -0.126   Prob(JB):                        0.606
Kurtosis:                       1.758   Cond. No.                     1.48e+04
==============================================================================
```

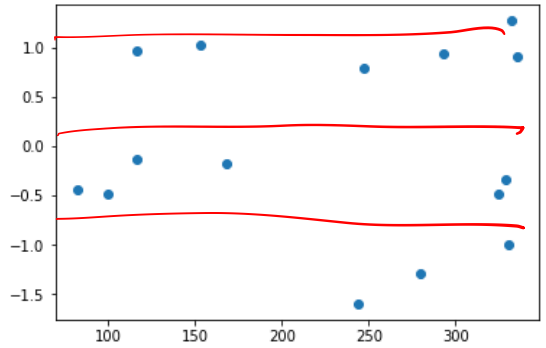
Second-order regression model

$$\text{Sales} = 45.3 + 6.34\,\text{Months} - .0345\,\text{MonthsSq}$$

MonthsSq = the square of the number of months the salesperson has been employed

Standardized residual plot for the Reynolds example: second-order model

```
In [35]: E2=Model2.resid_pearson
         E2

Out[35]: array([ 0.797777  , -0.99895952, -0.32984543,  1.27097898, -0.18118441,
                 0.97178443,  0.91436152, -0.48542046, -1.59531168, -1.28395183,
                -0.48348828, -0.13117488, -0.44045635,  0.94303218,  1.03185873])

In [38]: yhat2= Model2.predict(x_new2)

In [39]: plt.scatter(yhat2,E2)

Out[39]: <matplotlib.collections.PathCollection at 0x1609630dcc0>
```



Interpretation second order model

- Figure corresponding standardized residual plot shows that the previous curvilinear pattern has been removed.
- At the .05 level of significance, the computer output shows that the overall model is significant (*p*-value for the *F* test is 0.000)
- Note also that the *p*-value corresponding to the *t*-ratio for MonthsSq (*p*-value .002) is less than .05
- Hence we can conclude that adding MonthsSq to the model involving Months is significant.
- With an R-sq(adj) value of 88.6%, we should be pleased with the fit provided by this estimated regression equation.

Meaning of linearity in GLM

- In multiple regression analysis the word *linear* in the term "general linear model" refers only to the fact that $\beta_0, \beta_1, \ldots, \beta_p$ all have exponents of β_1
- It does not imply that the relationship between *y* and the *xi*'s is linear.
- Indeed, we have seen one example of how equation general linear model can be used to model a curvilinear relationship.

Thank you

IIT ROORKEE

NPTEL ONLINE CERTIFICATION COURSE

Regression Analysis Model Building (Interaction)- II

Dr. A. Ramesh

DEPARTMENT OF MANAGEMENT STUDIES

Agenda

- Incorporating Interaction of the independent variable to the regression model
- Python demo

Interaction

- If the original data set consists of observations for *y* and two independent variables *x*1 and *x*2, we can develop a second-order model with two predictor variables by setting $z_1 = x_1$, $z_2 = x_2$, $z_3 = x_1^2$, $z_4 = x_2^2$, and $z_5 = x_1x_2$ in the general linear model of equation
- The model obtained is

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1^2 + \beta_4 x_2^2 + \beta_5 x_1 x_2 + \epsilon$$

- In this second-order model, the variable $z_5 = x_1x_2$ is added to account for the potential effects of the two variables acting together.
- This type of effect is called **interaction**.

Example – Interaction

- A company introduces a new shampoo product.
- Two factors believed to have the most influence on sales are unit selling price and advertising expenditure.
- To investigate the effects of these two variables on sales, prices of $2.00, $2.50, and $3.00 were paired with advertising expenditures of $50,000 and $100,000 in 24 test markets.

Source: Statistics for Business and Economics,11th Edition by David R. Anderson (Author), Dennis J. Sweeney (Author), Thomas A. Williams (Author)

Price	Advertising Expenditure ($1000s)	Sales (1000s)
2	50	478
2.5	50	373
3	50	335
2	50	473
2.5	50	358
3	50	329
2	50	456
2.5	50	360
3	50	322
2	50	437
2.5	50	365
3	50	342
2	100	810
2.5	100	653
3	100	345
2	100	832
2.5	100	641
3	100	372
2	100	800
2.5	100	620
3	100	390
2	100	790
2.5	100	670
3	100	393

MEAN UNIT SALES (1000s)

		Price		
		$2.00	$2.50	$3.00
Advertising Expenditure	$50,000	461	364	332
	$100,000	808	646	375

Mean sales of 808,000 units when price = $2.00 and advertising expenditure = $100,000

Interpretation of interaction

- Note that the sample mean sales corresponding to a price of $2.00 and an advertising expenditure of $50,000 is 461,000, and the sample mean sales corresponding to a price of $2.00 and an advertising expenditure of $100,000 is 808,000.
- Hence, with price held constant at $2.00, the difference in mean sales between advertising expenditures of $50,000 and $100,000 is 808,000 - 461,000 = 347,000 units.

Interpretation of interaction

- When the price of the product is $2.50, the difference in mean sales is 646,000 -364,000 = 282,000 units.
- Finally, when the price is $3.00, the difference in mean sales is 375,000 - 332,000 = 43,000 units.
- Clearly, the difference in mean sales between advertising expenditures of $50,000 and $100,000 depends on the price of the product.
- In other words, at higher selling prices, the effect of increased advertising expenditure diminishes.
- These observations provide evidence of interaction between the price and advertising expenditure variables.

Importing Data

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm
```

In [8]:
```
tbl1 = pd.read_excel('Tyler.xlsx')
tbl1.head()
```

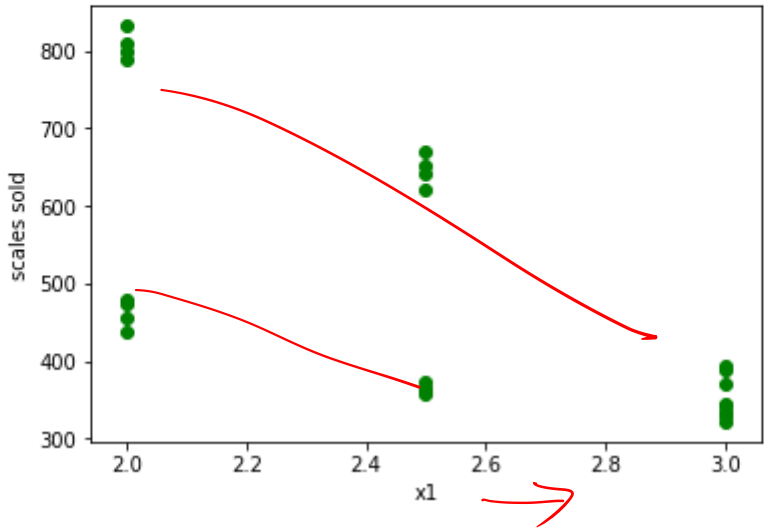
Out[8]:

	Price	AdvertisingExpenditure($1000s)	Sales(1000s)
0	2.0	50	478
1	2.5	50	373
2	3.0	50	335
3	2.0	50	473
4	2.5	50	358

Mean unit sales (1000s) as a function of selling price

```
In [7]: plt.scatter(tbl1['Price'],tbl1['Sales(1000s)'], color='green')
        plt.ylabel('scales sold')
        plt.xlabel('x1')
```

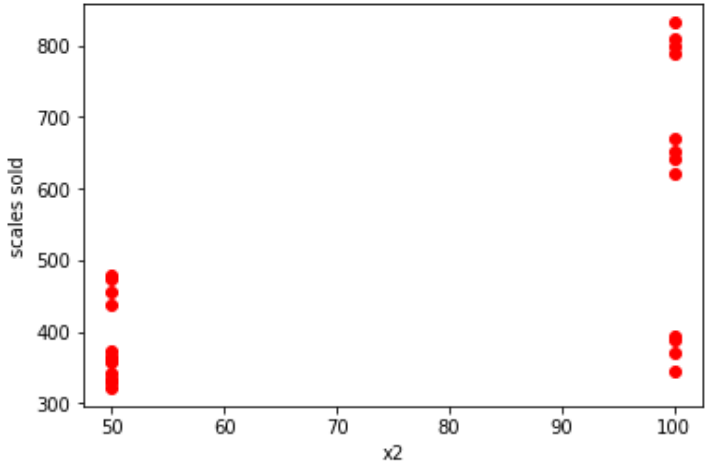
```
Out[7]: Text(0.5,0,'x1')
```



Mean unit sales (1000s) as a function of **Advertising Expenditure($1000s)**

```
In [6]: plt.scatter(tbl1['AdvertisingExpenditure($1000s)'],tbl1['Sales(1000s)'], color='red')
        plt.ylabel('scales sold')
        plt.xlabel('x2')
```

```
Out[6]: Text(0.5,0,'x2')
```



Need for study the interaction between variable

- When interaction between two variables is present, we cannot study the effect of one variable on the response *y* independently of the other variable.
- In other words, meaningful conclusions can be developed only if we consider the joint effect that both variables have on the response.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1 x_2 + \epsilon$$

y = unit sales (1000s)
x_1 = price ($)
x_2 = advertising expenditure ($1000s)

Estimated regression equation, a general linear model involving three independent variables (z_1, z_2, and z_3)

$$y = \beta_0 + \beta_1 z_1 + \beta_2 z_2 + \beta_3 z_3 + \epsilon$$

$$z_1 = x_1$$
$$z_2 = x_2$$
$$z_3 = x_1 x_2$$

Interaction variable

- *The data for the* PriceAdv *independent variable is obtained by multiplying each value of* Price *times the corresponding value of* AdvExp*.*

```
In [11]: z1 =tbl1['AdvertisingExpenditure($1000s)']
         z2 = tbl1['Price']
         z3 = z1*z2
```

New Model

In [12]:
```
x_new =np.column_stack((z1,z2,z3))
y = tbl1['Sales(1000s)']
xnew2 = sm.add_constant(x_new)
model2 = sm.OLS(y,xnew2)
Model2 = model2.fit()
print(Model2.summary())
```

OLS Regression Results

Dep. Variable:	Sales(1000s)	R-squared:	0.978
Model:	OLS	Adj. R-squared:	0.975
Method:	Least Squares	F-statistic:	297.9
Date:	Thu, 12 Sep 2019	Prob (F-statistic):	9.26e-17
Time:	13:12:52	Log-Likelihood:	-111.99
No. Observations:	24	AIC:	232.0
Df Residuals:	20	BIC:	236.7
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	-275.8333	112.842	-2.444	0.024	-511.218	-40.449
x1	19.6800	1.427	13.788	0.000	16.703	22.657
x2	175.0000	44.547	3.928	0.001	82.077	267.923
x3	-6.0800	0.563	-10.790	0.000	-7.255	-4.905

Omnibus:	0.641	Durbin-Watson:	2.842
Prob(Omnibus):	0.726	Jarque-Bera (JB):	0.565
Skew:	0.335	Prob(JB):	0.754
Kurtosis:	2.661	Cond. No.	4.53e+03

New Model

$$\text{Sales} = -276 + 175\,\text{Price} + 19.7\,\text{AdvExp} - 6.08\,\text{PriceAdv}$$

where

$$\begin{aligned}
\text{Sales} &= \text{unit sales (1000s)} \\
\text{Price} &= \text{price of the product (\$)} \\
\text{AdvExp} &= \text{advertising expenditure (\$1000s)} \\
\text{PriceAdv} &= \text{interaction term (Price times AdvExp)}
\end{aligned}$$

Interpretation

- Because the model is significant (*p*-value for the *F* test is 0.000) and the *p*-value corresponding to the *t* test for PriceAdv is 0.000, we conclude that interaction is significant given the linear effect of the price of the product and the advertising expenditure.
- Thus, the regression results show that the effect of advertising xpenditure on sales depends on the price.

Transformations Involving the Dependent Variable

$$y = b_0 + b_1 x_1 + b_2 x_2$$

$x_2 = 0, 1$

Miles per Gallon (y)	Weight (x)
28.7	2289
29.2	2113
34.2	2180
27.9	2448
33.3	2026
26.4	2702
23.9	2657
30.5	2106
18.1	3226
19.5	3213
14.3	3607
20.9	2888

Importing data

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm
```

In [2]:
```
tbl1 = pd.read_excel('MPG.xlsx')
tbl1
```

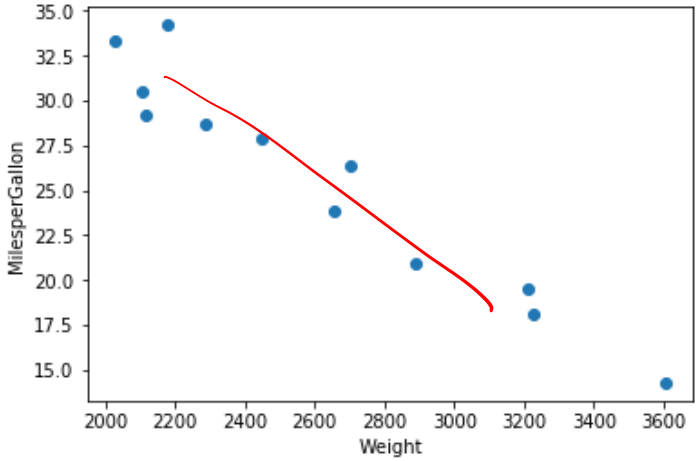
Out[2]:

	MilesperGallon	Weight
0	28.7	2289
1	29.2	2113
2	34.2	2180
3	27.9	2448
4	33.3	2026
5	26.4	2702
6	23.9	2657
7	30.5	2106
8	18.1	3226
9	19.5	3213
10	14.3	3607
11	20.9	2888

Scatter diagram

```
In [3]: plt.scatter(tbl1['Weight'],tbl1['MilesperGallon'])
        plt.ylabel('MilesperGallon')
        plt.xlabel('Weight')
```

```
Out[3]: Text(0.5,0,'Weight')
```



Model 1

```
In [4]: x =tbl1['Weight']
        y = tbl1['MilesperGallon']
        x2 = sm.add_constant(x)
        model = sm.OLS(y,x2)
        Model = model.fit()
        print(Model.summary())
```

```
C:\Users\Somi\Anaconda3\lib\site-packages\scipy\stats\stats.py:1394: UserWarning: kurtosistest only va
ing anyway, n=12
  "anyway, n=%i" % int(n))
```

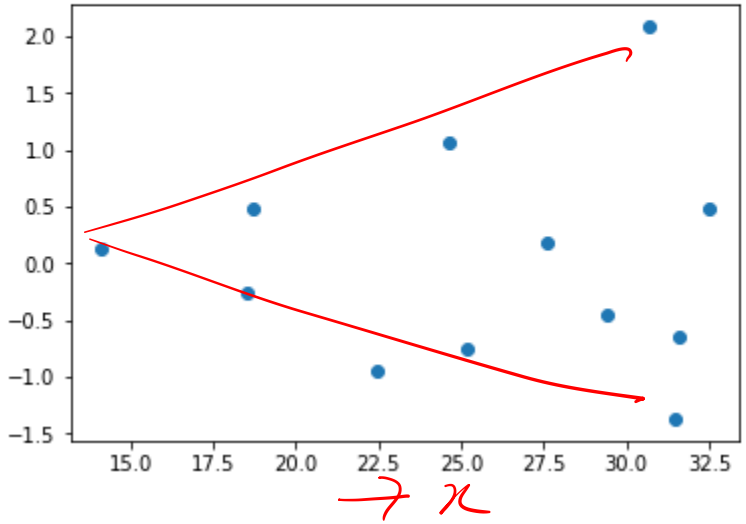
```
                            OLS Regression Results
==============================================================================
Dep. Variable:         MilesperGallon   R-squared:                       0.935
Model:                            OLS   Adj. R-squared:                  0.929
Method:                 Least Squares   F-statistic:                     144.8
Date:                Thu, 12 Sep 2019   Prob (F-statistic):           2.85e-07
Time:                        15:27:08   Log-Likelihood:                -22.091
No. Observations:                  12   AIC:                             48.18
Df Residuals:                      10   BIC:                             49.15
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         56.0957      2.582     21.725      0.000      50.342      61.849
Weight        -0.0116      0.001    -12.032      0.000      -0.014      -0.009
==============================================================================
Omnibus:                        2.266   Durbin-Watson:                   2.213
Prob(Omnibus):                  0.322   Jarque-Bera (JB):                0.951
Skew:                           0.690   Prob(JB):                        0.621
Kurtosis:                       3.025   Cond. No.                     1.43e+04
```

Standardized residual plot corresponding to the first-order model.

```
In [6]: E=Model.resid_pearson
        E

Out[6]: array([-0.44511273, -1.37252481,  2.08753315,  0.18422536,  0.47540179,
                1.05668329, -0.75350063, -0.64311699, -0.25953343,  0.4879158 ,
                0.12130227, -0.93927307])

In [7]: yhat = Model.predict(x2)
        yhat

Out[7]: 0     29.443573
        1     31.492839
        2     30.712721
        3     27.592247
        4     32.505829
        5     24.634783
        6     25.158743
        7     31.574344
        8     18.533557
        9     18.684924
        10    14.097361
        11    22.469081
        dtype: float64
```

Standardized residual plot corresponding to the first-order model

```
In [8]: plt.scatter(yhat,E)
Out[8]: <matplotlib.collections.PathCollection at 0x23f77072a58>
```



Model 2

In [12]:
```
Y = np.log(y)
```

In [13]:
```
model2 = sm.OLS(Y,x2)
Model2 = model2.fit()
print(Model2.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:         MilesperGallon   R-squared:                       0.948
Model:                            OLS   Adj. R-squared:                  0.942
Method:                 Least Squares   F-statistic:                     181.2
Date:                Thu, 12 Sep 2019   Prob (F-statistic):           9.84e-08
Time:                        15:34:13   Log-Likelihood:                 17.005
No. Observations:                  12   AIC:                            -30.01
Df Residuals:                      10   BIC:                            -29.04
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          4.5242      0.099     45.553      0.000       4.303       4.746
Weight        -0.0005   3.72e-05    -13.462      0.000      -0.001      -0.000
==============================================================================
Omnibus:                        0.899   Durbin-Watson:                   2.284
Prob(Omnibus):                  0.638   Jarque-Bera (JB):                0.779
Skew:                           0.484   Prob(JB):                        0.677
Kurtosis:                       2.211   Cond. No.                     1.43e+04
==============================================================================
```

Residual plot for model 2

```
In [14]: E2=Model2.resid_pearson
         E2
```

```
Out[14]: array([-0.31630114, -1.42005514,  1.5623004 ,  0.48370101, -0.0537228 ,
                 1.60448776, -0.29474869, -0.79674991, -0.18335787,  0.87474775,
                -0.87956572, -0.58073564])
```

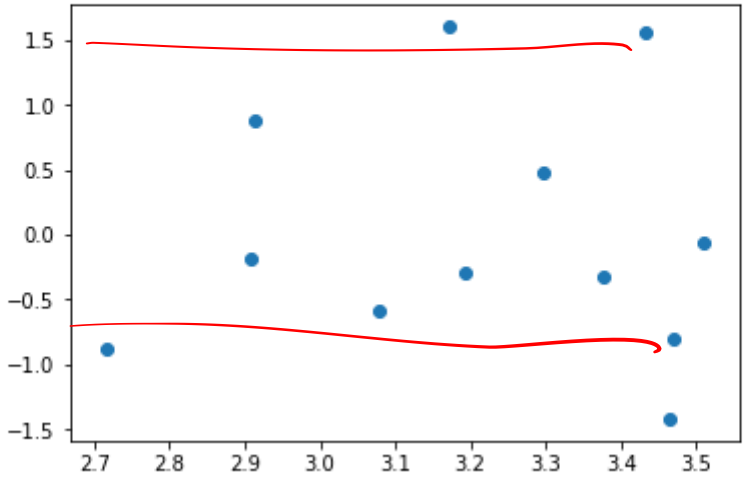
```
In [15]: yhat = Model2.predict(x2)
         yhat
```

```
Out[15]: 0     3.377221
         1     3.465414
         2     3.431840
         3     3.297547
         4     3.509009
         5     3.170268
         6     3.192817
         7     3.468922
         8     2.907694
         9     2.914208
         10    2.716776
         11    3.077064
         dtype: float64
```

Residual plot of model 2

```
In [16]: plt.scatter(yhat,E2)
```

```
Out[16]: <matplotlib.collections.PathCollection at 0x23f7737be10>
```



- The miles-per-gallon estimate is obtained by finding the number whose natural logarithm is 3.2675.
- Using a calculator with an exponential function, or raising *e* to the power 3.2675, we obtain 26.2 miles per gallon.

$$\text{LogeMPG} = 4.52 - 0.000501\,\text{Weight}$$

$$\text{LogeMPG} = 4.52 - 0.000501\,(2500) = 3.2675$$

Nonlinear Models That Are Intrinsically Linear

$$E(y) = \beta_0 \beta_1^x$$

$$E(y) = 500(1.2)^x$$

$$\log E(y) = \log \beta_0 + x \log \beta_1$$

$$y' = \log E(y), \beta_0' = \log \beta_0, \text{and } \beta_1' = \log \beta_1,$$

$$y' = \beta_0' + \beta_1' x$$

$$\hat{y}' = b_0' + b_1' x$$

Thank You